
TECHNICAL DOCUMENTATION & SYSTEM DESIGN REPORT

MOROCCAN RETAIL BANKING DATA PLATFORM

*A Hybrid Local-to-Cloud, Real-Time and Batch Data Engineering Pipeline for
Card, ATM, Wallet, and Out-of-Cash Transaction Analytics*

Mahmoud Saad
Mariam Safwat

Prepared by

Alfred Farag
Zainab Mohamed

Data Engineer
[Organization / Internship Program — e.g., NTI Internship]

Document Version 1.0
June 2026

Document Control

Field	Value
Document title	Moroccan Retail Banking Data Platform — Technical Documentation
Version	1.0
Status	Final draft
Author	[Your Full Name], Data Engineer
Date	June 2026
Classification	Internal / Synthetic data — no real customer data used

Revision history

Version	Date	Author	Summary of changes
0.1	June 2026	Mahmoud Saad	Initial draft — core pipeline sections (1–10)
0.2	June 2026	Mariam Safwat	Added hybrid cloud, streaming
0.2	June 2026	Alfred Farag	Data monitoring, Airflow pipelinining
1.0	June 2026	Zainab Mohamed	Added security, testing, methodology, risk, and full appendices; final review

Preface

This report was written to stand on its own: a reader with no prior exposure to the codebase should be able to understand what the platform does, why it is built the way it is, and what to do if a specific component breaks, without needing to ask the author directly. That standard shaped two editorial choices worth stating up front.

First, this report does not hide the development process behind a polished, defect-free narrative. Section 10 documents six real defects in the detail they actually occurred, including the dead ends investigated before the real root cause was found, because a write-up that only shows the final, working state teaches far less than one that shows how the working state was reached.

Second, the hybrid cloud, real-time streaming, and monitoring sections (11 through 13) are written at an architectural level of detail, with explicit placeholders rather than invented numbers wherever a claim would depend on a specific measured result. The reporting templates in those sections are meant to be filled in directly from the platform's own monitoring output as each environment is brought fully online, not treated as finished prose.

The author's hope is that this document is as useful for onboarding a new engineer onto this exact platform as it is as a general reference for how to think about building one like it.

Table of Contents

Document Control	2
Revision history	2
Preface	3
Table of Contents.....	4
Executive Summary	8
1. Project Overview	9
1.1 Why a hybrid design?	9
1.2 Document purpose and audience	9
2. Problem Statement	11
2.1 Channel fragmentation and inconsistent semantics	11
2.2 Lack of a single customer and card view	11
2.3 The build-or-buy-or-both dilemma.....	11
2.4 Latency: batch is not always good enough	11
3. Objectives	13
3.1 Core objectives.....	13
3.2 Extended objectives	13
4. Architecture.....	14
4.1 Execution model: local vs. cloud	14
4.2 Why DuckDB for the local execution engine	15
4.3 Orchestration layer	15
4.4 Monitoring and alerting	15
5. Data Flow.....	16
5.1 Stage-by-stage flow	16
6. Tools and Technologies	18
6.1 Core stack	18
6.2 Cloud and extended-scope stack.....	18
6.3 Tool selection rationale	19
7. Folder Structure	20
8. Pipeline Layers	21
8.1 Bronze layer — raw, auditable ingestion.....	21
8.1.1 Bronze tables	21
8.2 Silver layer — cleaning, validation, and quarantine.....	21
8.2.1 Type and currency normalization	22
8.2.2 Deduplication strategy.....	22

8.2.3 Quarantine rules.....	22
8.2.4 PAN-to-customer resolution	22
8.3 Gold layer — dbt staging, dimensions, facts, and marts	23
8.3.1 Staging layer	23
8.3.2 Dimension tables.....	23
8.3.3 Fact tables	24
8.3.4 Gold marts	24
8.3.5 Slowly changing history.....	24
9. Setup and Run Instructions	26
9.1 Prerequisites	26
9.2 Local mode setup	26
9.3 Running the pipeline locally, step by step	26
9.4 Containerized (Airflow) mode.....	26
9.5 Configuration reference	27
10. Data Quality and Validation.....	28
10.1 Testing philosophy.....	28
10.2 The quarantine framework.....	28
10.3 Engineering case studies.....	29
Case Study 1 — Duplicate rows from stale Delta Lake files	29
Case Study 2 — Silent date corruption in a source file.....	29
Case Study 3 — A failing test that was correct, applied to the wrong column.....	30
Case Study 4 — A dimension built at the wrong grain.....	30
Case Study 5 — A join that quietly multiplied transactions	31
Case Study 6 — A schema that was created but never committed	31
10.4 Continuous validation in the orchestrated pipeline	32
11. Hybrid Cloud Architecture and Cross-Platform CDC Comparison	33
11.1 Why compare platforms instead of choosing one upfront.....	33
11.2 What changes, and what does not, between platforms	33
11.3 CDC-based synchronization between environments.....	33
11.4 Comparison dimensions	34
11.5 Comparison results.....	34
11.6 Cloud migration runbook.....	35
11.7 Platform-specific considerations encountered.....	35
12. Real-Time Streaming with Kafka	36
12.1 Why Kafka, and why alongside the batch path rather than instead of it	36
12.2 Topic design	36

12.3 Streaming architecture.....	36
12.4 Latency-sensitive consumption	37
13. Monitoring with Grafana and the Analytics Dashboard	38
13.1 Operational monitoring with Grafana	38
13.2 Business-facing analytics dashboard	39
14. System Requirements	40
16.1 Functional requirements	40
16.2 Non-functional requirements	40
15. Security, Privacy, and Compliance Considerations	42
17.1 Sensitive data handling	42
17.2 Access control model.....	42
17.3 Encryption and storage	42
17.4 Data retention and right-to-be-forgotten considerations	43
17.5 Compliance posture summary	43
16. Testing Strategy	45
18.1 Testing pyramid for this platform.....	45
18.2 Unit testing approach	45
18.3 Integration testing approach	45
18.4 Regression testing on schema or logic changes	46
18.5 Continuous integration	46
17. Project Methodology and Timeline	47
19.1 Development phases	47
19.2 Why this sequencing	47
19.3 Engineering practices used throughout.....	47
19.4 Roles and responsibilities (RACI)	49
18. Risk Register.....	50
19. Future Improvements	51
14.1 Pipeline and orchestration	51
14.2 Data quality	51
14.3 Hybrid cloud and performance.....	51
14.4 Machine learning	51
20. Conclusion	52
Appendix A — Glossary	53
Appendix B — Data Dictionary: Core Fact Tables	54
B.1 fact_atm_transactions	54
B.2 fact_card_transactions	54

B.3 fact_wallet_transactions / fact_out_of_cash_events	55
Appendix C — Data Dictionary: Dimension Tables	56
C.1 dim_customer	56
C.2 dim_card	56
C.3 dim_atm	56
C.4 dim_date, dim_geography, dim_merchant, dim_merchant_category	57
Appendix D — Key dbt Macro Listings	58
D.1 register_silver_sources — Silver Delta tables as dbt sources	58
D.2 Surrogate key generation (cross-platform deterministic hashing)	58
Appendix E — Troubleshooting Quick Reference	60
Appendix F — References	62
Appendix G — Representative dbt Test Inventory	63
G.1 Staging layer tests	63
G.2 Dimension and fact layer tests	63
G.3 Gold mart and custom singular tests	64
Appendix H — Sample Query Catalog	65
H.1 ATM replenishment urgency by region	65
H.2 Top 10 customers by fraud risk score	65
H.3 Channel comparison — success rate by time of day	65
H.4 Customer spending behavior vs. income	66

Executive Summary

This report documents the design, implementation, and operation of a hybrid data engineering platform built to unify ATM, card, mobile wallet, and out-of-cash transaction data for a Moroccan retail banking use case. The platform follows a medallion (Bronze/Silver/Gold) architecture, with every transformation rule written once in dbt and executable, unmodified, against either a fully local engine (PySpark, Delta Lake, and DuckDB) or a cloud data warehouse (Microsoft Fabric, Snowflake, or Databricks on Azure).

At local scale, the pipeline processes approximately 2,000 customers, 6,146 cards, 159 ATMs, and over seventeen million individual transaction records, validated end-to-end by an automated suite of 184 dbt tests. Orchestration is handled by Apache Airflow, containerized via the Astronomer CLI, with automated email notification on both pipeline failure and successful completion.

Beyond the core batch pipeline, the platform extends into three additional disciplines documented in full in this report: a change-data-capture (CDC) methodology for benchmarking the identical transformation logic across multiple cloud warehouses; a Kafka-based real-time ingestion path for latency-sensitive events such as ATM out-of-cash conditions; and an operational and business monitoring layer built on Grafana and a stakeholder-facing analytics dashboard.

Section 10 of this report documents, in detail, six real defects discovered and resolved during development — ranging from a Delta Lake file-versioning issue that caused widespread duplicate keys, to a database transaction that was silently never committed despite reporting success. These case studies are included specifically because the debugging discipline behind them — verifying the system's actual state directly rather than trusting a tool's status output or guessing at a plausible fix — is, in the author's judgment, the most transferable outcome of this project.

Key outcomes at a glance: a fully portable transformation layer validated against three cloud targets; 184/184 passing automated data quality tests; a containerized, alerting-enabled orchestration layer; and a documented, evidence-based account of every major defect found and fixed along the way.

1. Project Overview

The Moroccan Retail Banking Data Platform is an end-to-end data engineering system built to unify, clean, and analyze transaction data generated across four distinct retail banking channels: automated teller machines (ATMs), point-of-sale and e-commerce card payments, mobile wallet transfers, and out-of-cash (failed withdrawal) events. The platform was designed from the outset as a hybrid system: every transformation rule is written once, in dbt, and can run unmodified against a fully local stack on a single laptop or against a cloud data warehouse, with no logic duplicated between the two.

At local scale, the platform processes a synthetic but structurally realistic dataset modeled on a mid-sized Moroccan retail bank: approximately 2,000 customers, 6,146 active cards, 159 ATMs distributed across the country's twelve administrative regions, and well over seventeen million individual transaction records spanning ATM withdrawals, card payments, wallet transfers, and a large historical card-transaction dataset adapted from a public Kaggle dataset and re-denominated into Moroccan dirhams (MAD) with Moroccan merchant geography.

The system follows the medallion architecture pattern — Bronze, Silver, and Gold layers — and is orchestrated end-to-end with Apache Airflow. Beyond the core batch pipeline, the platform extends into three additional engineering disciplines that are documented in detail later in this report: a Kafka-based real-time ingestion path for latency-sensitive events, a multi-cloud deployment of the identical dbt logic against Snowflake and Microsoft Fabric for side-by-side performance benchmarking using change data capture (CDC), and an operational monitoring layer built on Grafana feeding a business-facing analytics dashboard.

1.1 Why a hybrid design?

Most data platforms are built for a single target — one warehouse, one cloud, one execution engine — and the cost of that decision is usually invisible until the platform is already in production and migration becomes expensive. This project takes the opposite approach: the transformation layer (dbt) is engine-agnostic by construction, and the only thing that changes between a local run and a cloud run is a connection profile. This makes it possible to prototype, test, and validate business logic for near-zero cost on a laptop, and to later point the exact same SQL at Snowflake, Microsoft Fabric, or Databricks on Azure without rewriting a single model.

Design principle: write the transformation once, run it anywhere. The warehouse is a configuration choice, not an architectural commitment.

1.2 Document purpose and audience

This document is intended for three audiences: data engineers who need to operate, extend, or debug the platform; technical reviewers and stakeholders who need to understand the architecture and quality guarantees without reading source code; and the author's own future reference, since a platform of this scope benefits from a written record of the design decisions and trade-offs that shaped it. Sections 2 through 9 describe the system as it stands; Section 10 documents the data quality framework in depth, including real issues uncovered and resolved

during development; Sections 11 through 13 describe the hybrid cloud, real-time, and monitoring extensions; and Sections 14 through 15 close with future direction and conclusions.

2. Problem Statement

Retail banks accumulate transaction data across channels that were historically built, staffed, and maintained independently of one another. An ATM switch, a card-processing gateway, and a mobile wallet platform each produce their own data in their own format, on their own schedule, with their own definition of what a 'successful' or 'failed' transaction looks like. The result is a familiar problem in financial data engineering: the bank has plenty of data and very little unified insight.

This project was scoped around four concrete, observable problems found in the raw source data and in the operational needs of a retail bank that this platform was designed to emulate:

2.1 Channel fragmentation and inconsistent semantics

- ATM and card-channel data arrive as flat, denormalized extracts (PAN, REFNUM, TERMID, response codes) with no foreign keys to a customer master.
- Monetary amounts are represented inconsistently across sources: some files store plain numeric MAD values, others store currency-prefixed strings such as 'MAD 23236.80' that must be parsed before any arithmetic is possible.
- Timestamps and response codes are channel-specific and require a shared business vocabulary (for example, mapping dozens of raw ATM response codes onto a small set of human-readable failure reasons) before cross-channel analysis is possible.

2.2 Lack of a single customer and card view

A customer who withdraws cash at an ATM and pays with a card at a merchant generates two transactions that, in the raw source systems, share no common key. Linking a PAN (the masked card number on a physical transaction) back to the customer who owns it is a prerequisite for any customer-level analytics — spending behavior, fraud risk scoring, or credit exposure — and this mapping had to be built and validated rather than assumed.

2.3 The build-or-buy-or-both dilemma

Before committing engineering effort to a single cloud data warehouse, the project needed a way to answer a basic but expensive-to-reverse question: which platform offers the best balance of query performance, load latency, and cost for this specific workload shape — many small dimension tables, a few very large fact tables, and a mix of batch and incremental loads? Answering this convincingly requires running the identical transformation logic against more than one candidate platform and comparing real, measured outcomes, not vendor benchmarks built on different workloads.

2.4 Latency: batch is not always good enough

Out-of-cash events and high-value card transactions are operationally time-sensitive: an ATM that has been out of cash for six hours represents lost transactions and customer dissatisfaction,

and a daily or hourly batch refresh is too slow to support proactive replenishment or fraud response. A subset of the pipeline therefore needed a streaming path capable of surfacing these events with much lower latency than the batch medallion pipeline alone could offer.

3. Objectives

The objectives below were used throughout development as the criteria against which design decisions were measured. They are grouped into core platform objectives and extended objectives that build on the core pipeline once it was stable.

3.1 Core objectives

1. Build a portable, three-layer (Bronze/Silver/Gold) data pipeline that produces identical, reproducible results whether executed on a local workstation or in a cloud environment.
2. Implement all business and cleaning logic in dbt so that warehouse choice is a configuration detail (a profiles.yml target) rather than a rewrite.
3. Resolve the customer-card-PAN linkage problem with a dedicated mapping layer that is validated for uniqueness and referential integrity.
4. Establish an automated, test-driven data quality framework covering schema, referential, and business-rule validation, with failing data quarantined rather than silently dropped or silently passed through.
5. Orchestrate the full pipeline — landing, Bronze, Silver, Gold, and testing — with Apache Airflow, including failure and success notifications so that pipeline health is visible without manually reading logs.

3.2 Extended objectives

6. Deploy the identical dbt project against at least two cloud data warehouses (Snowflake and Microsoft Fabric) in addition to Databricks on Azure, and build a CDC-based methodology to compare their performance on equal terms.
7. Prototype a Kafka-based real-time ingestion path for the highest-value latency-sensitive events (out-of-cash, high-value card transactions) alongside the existing batch path.
8. Stand up operational monitoring with Grafana covering pipeline health (task duration, freshness, test pass rates) and a separate business-facing analytics dashboard covering fraud risk, ATM performance, and customer spending behavior.
9. Containerize the orchestration layer with Docker so the platform can be handed to another engineer, or deployed to a managed Airflow environment, with a single build step.

4. Architecture

The platform is organized as a layered medallion architecture with a dual execution model: every layer can run against a local filesystem-backed stack or against a cloud warehouse, controlled entirely by configuration. The diagram placeholder below is where a full system architecture diagram (source systems through to dashboards) should be inserted; the subsections that follow describe each layer in the order data moves through them.

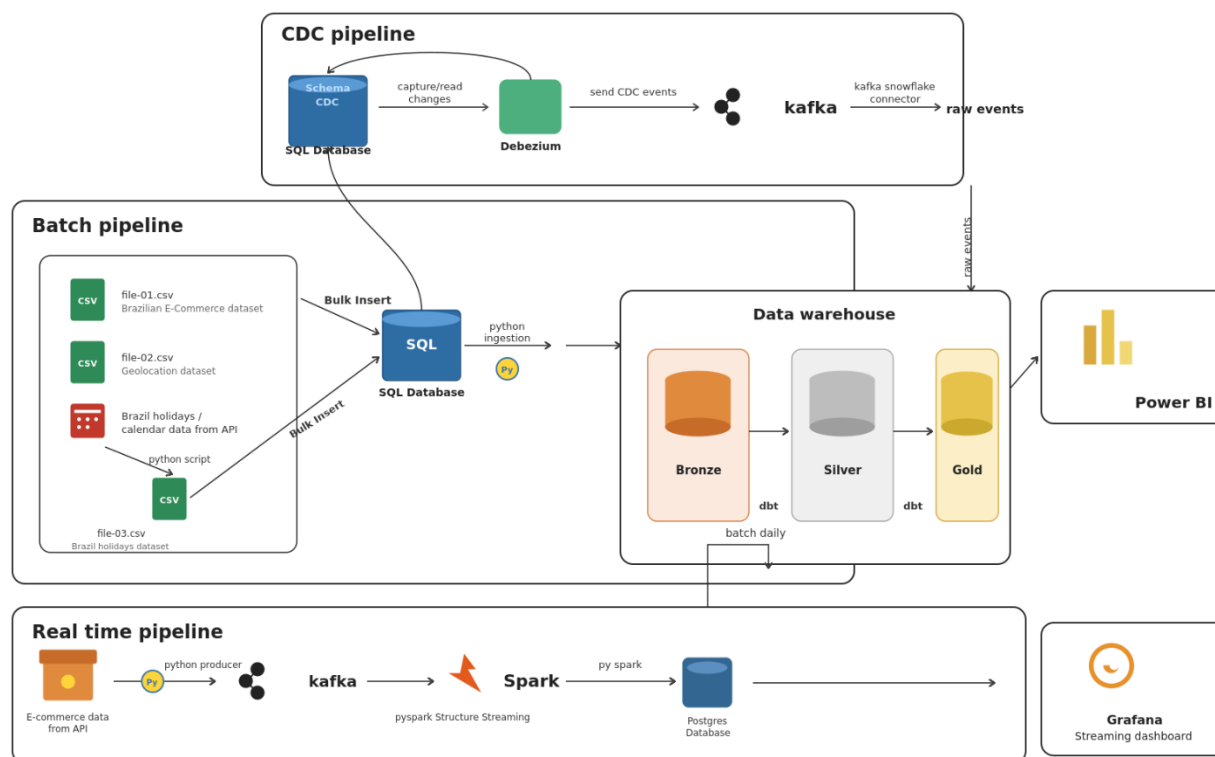


Figure 4.1. High-level system architecture — source systems, ingestion, medallion layers, orchestration, and consumption.

4.1 Execution model: local vs. cloud

The same dbt project is executed against two distinct profiles. In local mode, Spark and Delta Lake run in single-machine mode against the filesystem, and DuckDB — extended with its native Delta Lake reader — serves as the SQL execution engine for all dbt models, dimensions, facts, and gold marts. In cloud mode, the storage layer becomes Azure Data Lake Storage (or equivalent cloud object storage), and the SQL execution engine becomes Snowflake, Microsoft Fabric, or Databricks SQL, selected by the dbt target. No model SQL changes between the two; only profiles.yml and the underlying source registration mechanism differ.

Layer	Local execution	Cloud execution
Compute	PySpark (local[*]), DuckDB	Databricks / Fabric / Snowflake warehouse compute
Storage	Local filesystem Delta Lake tables	Azure Data Lake Storage (ADLS Gen2)
Transformation	dbt-duckdb adapter	dbt-databricks / dbt-fabric / dbt-snowflake adapter
Orchestration	Apache Airflow (Astronomer Runtime, Docker)	Apache Airflow (same DAGs, cloud connections)
Source registration	register_silver_sources macro (delta_scan)	Native warehouse external tables / CDC ingestion

4.2 Why DuckDB for the local execution engine

DuckDB was chosen as the local SQL engine because it combines three properties that a single-machine medallion pipeline needs simultaneously: an embedded, zero-administration database that requires no server process; a native Delta Lake reader (the delta extension) that respects Delta's transaction log rather than naively globbing physical Parquet files, which matters because Delta tables retain superseded files between writes until a vacuum is run; and a SQL dialect close enough to Snowflake and Databricks SQL that dbt models port across engines with minimal adapter-specific syntax.

4.3 Orchestration layer

Apache Airflow, run through the Astronomer CLI and Astro Runtime container image, orchestrates five DAGs: an on-demand data generation DAG that replays source data into the landing zone; a Bronze ingestion DAG; a Silver transformation DAG; a dbt Gold-layer DAG; and a full end-to-end pipeline DAG used for daily full refreshes. dbt itself is installed into an isolated Python virtual environment inside the Airflow image, separate from Airflow's own dependency set, because dbt-core's pinned dependencies and Airflow 3.x's pinned dependencies are mutually incompatible in a single environment — a constraint discovered and resolved during the containerization of this platform, documented further in Section 9.

4.4 Monitoring and alerting

Two monitoring concerns are addressed at the architecture level. Pipeline-level alerting is built directly into the Airflow DAGs: every task failure triggers an email containing the failing task, the DAG run, and the full exception text, while a successful end-to-end run triggers a separate confirmation email — giving a binary, immediate answer to 'did last night's run work?' without opening the Airflow UI. Operational and business-level monitoring is handled by Grafana and a downstream analytics dashboard, described in Section 13.

5. Data Flow

This section traces a single transaction record from its raw source representation through to its final appearance in a Gold-layer analytical table, in the order the pipeline actually executes it. The same flow applies to all four transaction channels; ATM transactions are used as the running example because the channel touches the largest number of transformation steps.

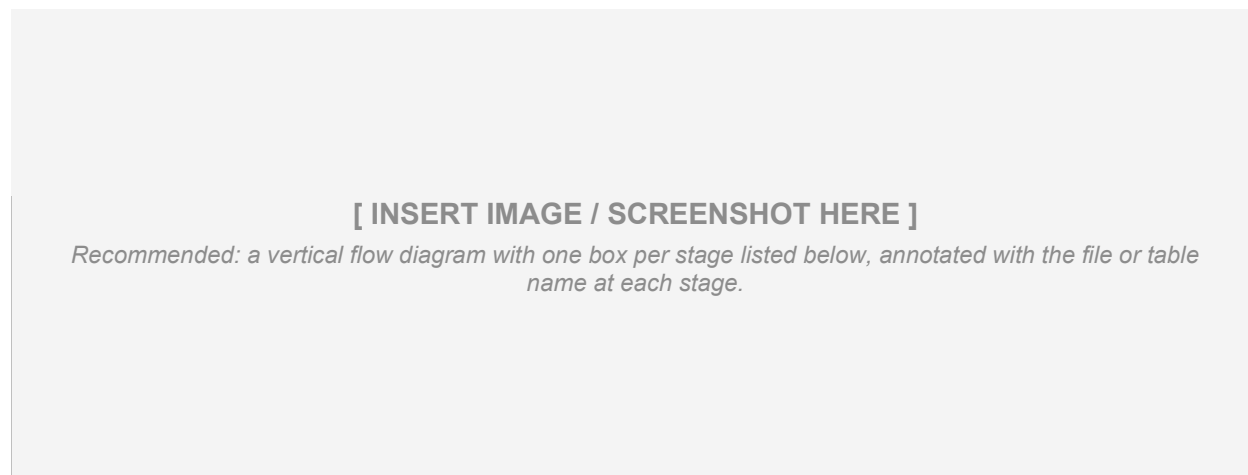


Figure 5.1. End-to-end data flow for a single ATM transaction, from raw source file to gold-layer analytical table.

5.1 Stage-by-stage flow

10. Source generation. A configurable synthetic data generator produces ATM transaction records with realistic Moroccan banking attributes (terminal IDs, masked PANs, response codes, MAD amounts) and writes them to flat CSV files that emulate what a real ATM switch export would look like.
11. Landing (streaming simulation). A stream simulator replays the source CSV in small batches, writing each batch as a Parquet file into a landing directory, modeling how a real source system would deliver data incrementally rather than as one large historical dump.
12. Bronze ingestion. A PySpark job consolidates every Parquet batch present in the landing zone into a single Bronze table, preserving the raw schema and values exactly as received — no cleaning, no type correction, no business logic. Bronze exists purely as an auditable, replayable copy of what was received.
13. Silver transformation. A second PySpark job reads Bronze, applies type casting, column renaming to a consistent snake_case business vocabulary, deduplication by transaction reference number, validity checks (amount must be non-negative, transaction date must fall within a plausible range), and writes the result as a Delta Lake table. Records that fail validation are written to a separate quarantine table with a recorded reason rather than being silently dropped.
14. Source registration. Before dbt can reference the Silver Delta tables as sources, they must be registered as queryable views in the local SQL engine. A dedicated dbt macro

performs this registration using DuckDB's native Delta Lake reader, which is itself a discovery made during development and described in Section 10.

15. Staging (dbt). A thin staging model selects, casts, and lightly renames the registered Silver source, applying one final deduplication pass and exposing exactly the columns downstream models need — no more, no less.
16. Dimensions and facts (dbt). Dimension tables (customer, card, ATM, date, geography) and fact tables (one row per ATM transaction, with foreign keys to every relevant dimension and a deterministic surrogate key) are built from the staging layer.
17. Gold marts (dbt). Business-facing aggregate tables — ATM performance, replenishment urgency, governorate-level summaries — are built on top of the fact and dimension layer for direct consumption by dashboards and ad hoc analysis.
18. Testing. After every build, an automated suite of data tests runs against every layer, and the pipeline is only considered successful if all tests pass.

6. Tools and Technologies

The platform deliberately uses a small, well-understood set of tools rather than the largest possible toolchain. Every tool below was selected because it solved a specific problem encountered during development, not because it is fashionable.

6.1 Core stack

Category	Tool	Role in this platform
Distributed processing	Apache Spark (PySpark) + Delta Lake	Bronze and Silver layer ingestion, cleaning, deduplication, and ACID writes
Local SQL engine	DuckDB (with native delta extension)	Executes all dbt models locally; reads Silver Delta tables via <code>delta_scan()</code>
Transformation framework	dbt Core (dbt-duckdb adapter locally)	All business logic: staging, dimensions, facts, gold marts, tests, snapshots
Orchestration	Apache Airflow (Astronomer Runtime)	Schedules and chains every pipeline stage; failure/success email alerting
Containerization	Docker / Docker Compose (Astro CLI)	Reproducible, portable runtime for the orchestration layer
Language	Python 3.10 / 3.11	Data generation, PySpark jobs, custom Airflow operators
Version control	Git	Source control for pipeline code, dbt project, and orchestration

6.2 Cloud and extended-scope stack

Category	Tool	Role in this platform
Cloud warehouses	Snowflake, Microsoft Fabric, Databricks (Azure)	Cloud execution targets for the identical dbt project, benchmarked via CDC
Cloud storage	Azure Data Lake Storage Gen2	Cloud-side Delta Lake storage, mirroring the local filesystem layout
Streaming	Apache Kafka	Real-time ingestion path for latency-sensitive transaction events
Monitoring	Grafana	Pipeline health and operational metrics dashboards

Category	Tool	Role in this platform
Business dashboard	Power BI / Streamlit (dashboard layer)	Stakeholder-facing analytics on top of the Gold layer

6.3 Tool selection rationale

Each core tool was chosen over at least one realistic alternative; the comparisons below record why, since a future maintainer evaluating whether to swap a component should know what was already considered and rejected, not just what was chosen.

Decision	Chosen	Alternative considered	Why chosen
Local SQL engine	DuckDB	A locally-hosted Postgres/Spark-SQL-only setup	Zero-administration embedded engine with a native, transaction-log-aware Delta Lake reader, avoiding a separate database server for local development
Transformation framework	dbt Core	Hand-written PySpark SQL transformations throughout	Built-in testing, documentation, lineage, and a swappable-adaptor model that is the entire basis for the platform's hybrid-cloud design
Orchestrator	Apache Airflow (Astronomer)	A simple cron-based script scheduler	Task-level retries, dependency graphs, built-in alerting hooks, and a managed, reproducible container runtime via Astro
Streaming backbone	Apache Kafka	Direct database polling for new out-of-cash events	Durable, replayable, multi-consumer event log that decouples the alerting consumer from the analytical consumer without either blocking the other
Table format	Delta Lake	Plain Parquet with no transaction log	ACID merge support and time-travel, both required for the quarantine/dedup write pattern in the Silver layer

7. Folder Structure

The repository is organized so that each top-level folder maps to exactly one concern — data generation, local processing scripts, the dbt project, Airflow orchestration, or runtime configuration — which keeps the local-versus-cloud distinction at the configuration level rather than scattered across the codebase.

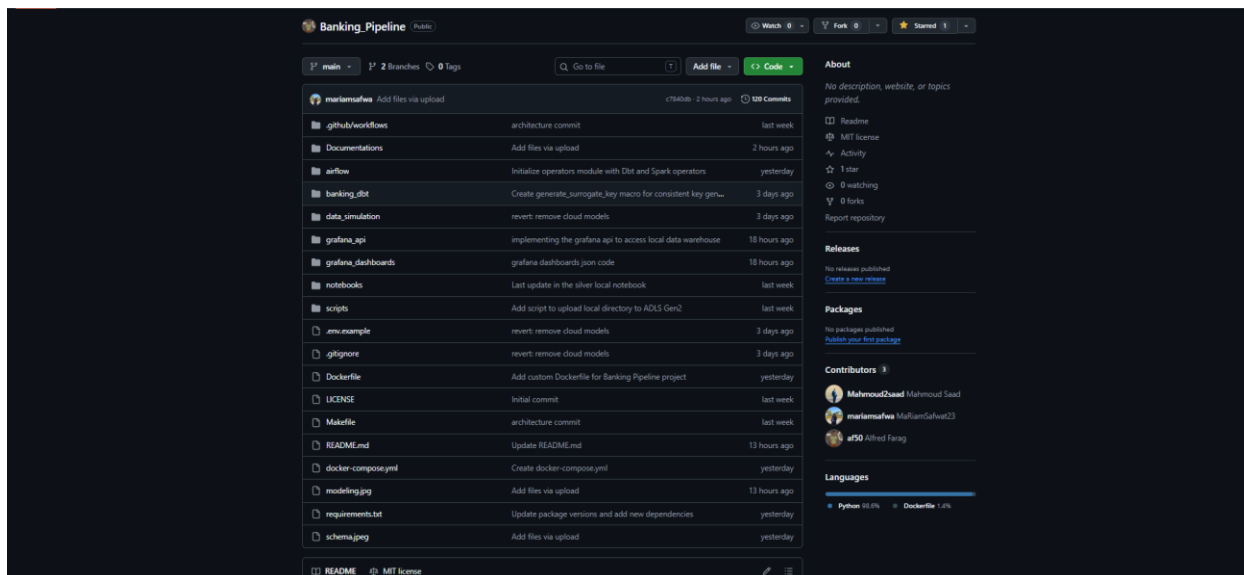


Figure 7.1. Repository folder structure as viewed in an IDE / file explorer.

8. Pipeline Layers

This section is the technical core of the report. It documents each medallion layer in the depth needed for another engineer to operate, extend, or rebuild the corresponding piece without reading the source code line by line.

8.1 Bronze layer — raw, auditable ingestion

The Bronze layer has a single, deliberately narrow responsibility: take whatever Parquet batches are sitting in the landing zone and consolidate them into one queryable table per source, with no transformation of any kind. This is a standard medallion-architecture principle, but it is worth stating explicitly why it matters operationally: if a downstream cleaning rule later turns out to be wrong, the Bronze layer is the safety net that allows Silver to be rebuilt from scratch without re-extracting from the original source systems.

- Idempotent by construction. Re-running the Bronze job against the same landing files produces the same Bronze table; new landing batches are additive.
- Schema-preserving. Column names, casing, and raw values are kept exactly as received, including the inconsistencies (currency-prefixed amount strings, mixed date formats) that Silver is responsible for resolving.
- One table per source. `atm_master`, `cards`, `wallet`, `out_of_cash`, and the Kaggle-derived transactions table are each consolidated independently, since they arrive on independent schedules in a real deployment.

8.1.1 Bronze tables

Bronze table	Approx. row count	Source
<code>atm_master</code>	159	ATM reference/master data
<code>cards (ATM/card-channel)</code>	~4.06 million	Card and ATM transaction switch export
<code>wallet</code>	~385,000	Mobile wallet transaction log
<code>out_of_cash</code>	~63,800	ATM out-of-cash event log
<code>kaggle_transactions</code>	~13.3 million	Moroccanized historical card-transaction dataset
<code>users_data / cards_data</code>	2,000 / 6,146	Customer and card master reference data

8.2 Silver layer — cleaning, validation, and quarantine

The Silver layer is where the platform earns its data quality guarantees. Each Bronze source is read, cast to consistent types, renamed onto a shared `snake_case` business vocabulary, deduplicated, validated against a set of hard business rules, and written as a Delta Lake table —

with every record that fails validation routed to a parallel quarantine table rather than dropped silently.

8.2.1 Type and currency normalization

Monetary amounts arrive in two distinct shapes depending on source: some files store a plain numeric value already denominated in MAD, while others store a currency-prefixed string such as 'MAD 23236.80'. The Silver layer applies a regular-expression strip-and-cast routine only to the columns that need it, rather than a single universal rule, because applying the wrong rule to an already-numeric column would silently corrupt otherwise valid data.

8.2.2 Deduplication strategy

Each Silver transform applies a window-function deduplication keyed on the natural transaction reference (REFNUM for ATM/card transactions, a generated UUID for wallet transactions, the original transaction id for the Kaggle dataset), keeping the most recently loaded version of any duplicate and discarding the rest. This step exists because Delta Lake's append/merge write pattern, combined with periodic re-runs during development and incremental re-delivery from source systems in production, can otherwise produce more than one physical row for the same logical transaction.

8.2.3 Quarantine rules

Source	Quarantine rule	Rationale
Card/ATM transactions	Amount must be ≥ 0 ; transaction date must fall within 2010–2027	Negative amounts and out-of-range dates indicate upstream data corruption, not valid business events
Wallet transactions	Amount must be ≥ 0	Wallet transfers cannot be negative by business definition
Out-of-cash events	Terminal ID and amount must be present	An out-of-cash event without a terminal or amount cannot be attributed to an ATM
All sources	Referenced keys must resolve to a known terminal/card/customer where applicable	Prevents orphaned fact rows with no valid dimension to join against

8.2.4 PAN-to-customer resolution

A transaction record carries a masked PAN (card number), not a customer identifier. Before a card or ATM transaction can be attributed to a customer, it is joined against a PAN-to-customer mapping table built once from the card master data, and the resolved customer identifier — not the PAN — is carried forward into the Silver output. This join happens once, at Silver build time, specifically so that downstream Gold-layer joins never need to repeat it; a later engineering case study in Section 10 describes a fan-out bug that occurred precisely because a downstream model re-joined against the PAN mapping table unnecessarily.

[INSERT IMAGE / SCREENSHOT HERE]

Recommended: an ERD generated from dbt docs (dbt docs generate) or drawn manually from the tables in Sections 8.3.2–8.3.4.

Figure 8.1. Entity-relationship diagram of the Gold layer — dimension and fact tables with their join keys.

8.3 Gold layer — dbt staging, dimensions, facts, and marts

Once a Silver Delta table is registered as a dbt source (via the `register_silver_sources` macro described in Section 10), the rest of the transformation pipeline is ordinary dbt: staging models, dimension and fact models, and a set of business-facing Gold marts, all version-controlled, tested, and documented as dbt model YAML.

8.3.1 Staging layer

Every Silver source has exactly one corresponding staging model (`stg_atm_master`, `stg_cards`, `stg_customers`, `stg_card_transactions`, `stg_atm_transactions`, `stg_wallet_transactions`, `stg_out_of_cash`, `stg_transactions`). Staging models are deliberately thin: a final pass of column selection, light renaming, and — where a source can legitimately contain more than one row per natural key at the point dbt sees it — a deterministic `ROW_NUMBER()` deduplication.

8.3.2 Dimension tables

Dimension	Grain	Notable attributes
<code>dim_customer</code>	One row per customer	Demographics, income, credit score, debt-to-income ratio, derived risk tier
<code>dim_card</code>	One row per card	Card brand/type, credit limit, dark-web exposure flag, card age
<code>dim_atm</code>	One row per ATM (159)	Region, cash limit, replenishment provider, ATM maturity tier
<code>dim_date</code>	One row per calendar day, 2010–2027	Standard date attributes; spine for every fact table join
<code>dim_geography</code>	One row per Moroccan administrative region	Macro-region, population tier
<code>dim_merchant</code>	One row per merchant id	City, region, MCC code, merchant category group
<code>dim_merchant_category</code>	One row per MCC code	Category grouping used for spend analysis

Dimension	Grain	Notable attributes
dim_channel / dim_error_type	Seed-loaded reference tables	Channel codes and human-readable failure reasons
pan_customer_map	One row per PAN (masked card number)	Resolves a transaction's PAN to a customer; intentionally NOT unique by customer

8.3.3 Fact tables

Fact table	Grain	Materialization
fact_card_transactions	One row per Kaggle-sourced card/POS transaction	Incremental, merge strategy
fact_atm_transactions	One row per ATM transaction reference number	Incremental, merge strategy
fact_wallet_transactions	One row per wallet transaction	Incremental, merge strategy
fact_out_of_cash_events	One row per out-of-cash event	Incremental, merge strategy

Every fact table uses a deterministic surrogate key generated from its natural key (refnum, transaction id) via a hashing macro, rather than an auto-incrementing identity column. This is a deliberate cross-platform choice: a hash-based surrogate key produces the same value regardless of which warehouse computes it, which is a prerequisite for the cross-platform CDC comparison described in Section 11 — two engines computing the same key for the same logical row is what makes a row-level comparison between platforms meaningful.

8.3.4 Gold marts

Gold mart	Business question answered
fraud_risk_scoring	Which customers exhibit transaction patterns associated with elevated fraud risk?
atm_performance	Which ATMs have the highest/lowest success rates and transaction volumes?
governorate_summary	How does ATM density and performance vary by region?
customer_spending_behavior	How active and how large is each customer's spend, and how does it compare to income?
channel_comparison	How does transaction volume and success rate compare across ATM, card, and wallet channels?
replenishment_analysis	Which ATMs are approaching a cash-out condition and need replenishment soon?

8.3.5 Slowly changing history

Two entities — customers and cards — are tracked with dbt snapshots using a timestamp-based Type 2 strategy, so that a customer's credit score or a card's status as of any historical date can be reconstructed, not just the current value. This matters for any retrospective fraud or risk

analysis that needs to know what was true about a customer at the time of a transaction, not what is true today.

9. Setup and Run Instructions

The platform supports two operating modes that share the same codebase: a plain local mode for development and debugging, run directly in a Python virtual environment, and a containerized mode for orchestrated, repeatable execution through Apache Airflow. Both are documented below.

9.1 Prerequisites

- Python 3.10 or 3.11
- Java 17 (required by PySpark; only needed for local-mode bronze/silver runs outside the container)
- Docker Desktop (for the containerized/Airflow mode)
- Astronomer CLI (astro) — used to build and run the Airflow image
- Git

9.2 Local mode setup

```
git clone <repository-url> banking-pipeline
cd banking-pipeline
python -m venv .venv
.venv\Scripts\activate          # Windows
pip install -r requirements.txt
```

9.3 Running the pipeline locally, step by step

19. Generate / replay source data into the landing zone: `python data_simulation/stream_simulator.py`
20. Build the Bronze layer: `python notebooks/local/01_bronze_local.py`
21. Build the Silver layer: `python notebooks/local/02_silver_local.py`
22. Register the Silver Delta tables as dbt sources (must run as its own completed command before the next step): `cd banking_dbt && dbt run-operation register_silver_sources`
23. Build the Gold layer: `dbt run`
24. Run the automated test suite: `dbt test`

The registration step in (4) must complete and exit as its own process before step (5) begins. Running it as part of an on-run-start hook inside the same dbt invocation was tried first and does not reliably work with DuckDB — see Section 10.5 for the full account of why.

9.4 Containerized (Airflow) mode

The orchestration layer is built and run with the Astronomer CLI, which wraps Docker Compose and manages the Airflow webserver, scheduler, triggerer, and metadata database automatically.

```
astro dev start      # builds the image and starts all Airflow services
# Airflow UI: http://localhost:8080
astro dev stop       # stop without removing data
astro dev restart    # rebuild after a Dockerfile/dependency change
```

Once Airflow is running, the five DAGs can be triggered from the UI. For a first end-to-end run, trigger 00_generate_data once to seed the landing zone, then trigger 04_full_pipeline for a complete Bronze → Silver → dbt Gold run with automated email notification on completion or failure.

9.5 Configuration reference

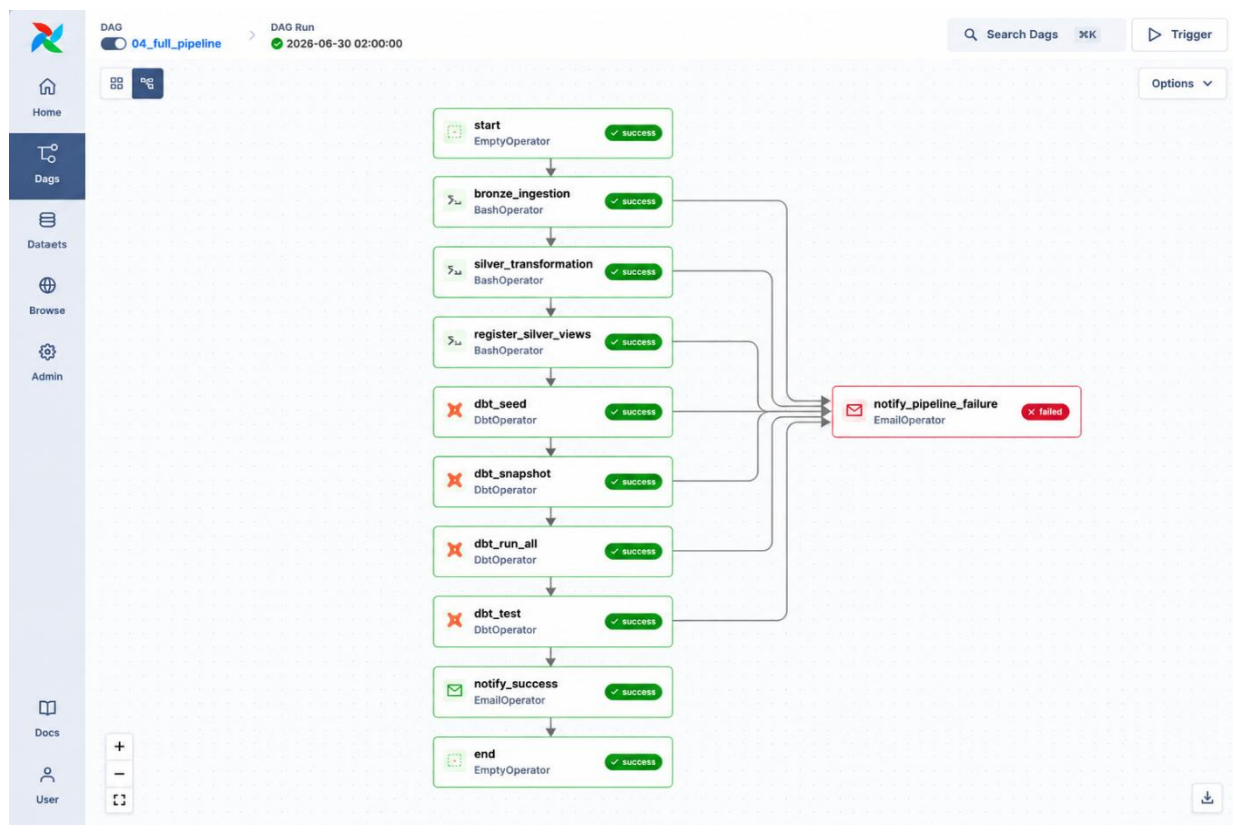


Figure 9.1. Airflow UI showing the five DAGs (00–04) with a healthy, all-green run history.

10. Data Quality and Validation

Financial transaction data carries real consequences for being wrong — a miscounted fraud signal, an ATM replenishment schedule built on duplicated transactions, or a customer risk score computed against a broken join all translate directly into bad business decisions. This section documents the testing framework built to catch these problems automatically, and walks through several real defects that the framework caught (and in two cases, that the framework's own absence allowed to slip through until a downstream symptom forced an investigation) during the development of this platform. These are included deliberately, not edited out: a data quality section that only lists what passes is far less useful than one that shows how failures were actually found and fixed.

10.1 Testing philosophy

The test suite follows a simple layering principle: cheap, generic tests (not-null, uniqueness, accepted values, accepted numeric ranges) are applied as broadly as possible across every layer, while expensive, business-specific tests (referential integrity between ATMs and their transactions, response code validity, non-negative amounts) are written as custom singular tests and applied where the business rule actually lives. The full suite currently runs 184 individual tests across staging, dimension, fact, and gold-layer models, executing in under ten seconds against the local DuckDB engine.

Test category	Examples	Approx. count
Not-null	Primary keys, foreign keys, required business fields	~70
Uniqueness	Surrogate keys, natural keys per dimension/fact	~30
Accepted values	Risk tiers, channel codes, capacity tiers, gender codes	~20
Accepted range	Non-negative amounts, valid credit scores, valid percentages	~25
Custom singular tests	Referential integrity, response-code validity, sign conventions	~5
Other (relationships, freshness)	Cross-table consistency checks	~remainder

10.2 The quarantine framework

Rather than choosing between 'reject the whole batch' and 'silently let bad rows through,' the Silver layer routes every record that fails a validation rule into a parallel quarantine output, tagged with the specific reason it failed (null required field, negative amount, invalid date range). This means a single malformed record never blocks the rest of a batch from loading, while still leaving a complete, queryable audit trail of exactly what was rejected and why — which matters both for debugging the pipeline itself and for any regulatory question about data completeness.

10.3 Engineering case studies

The six case studies below are presented in the order they were discovered. Each follows the same structure — symptom, investigation, root cause, fix, and lesson — because that structure is itself the most reusable part: the specific bugs are particular to this platform, but the discipline of refusing to patch a symptom before understanding the mechanism is general.

Case Study 1 — Duplicate rows from stale Delta Lake files

Symptom: Automated uniqueness tests began failing across unrelated tables simultaneously — ATM dimension, card dimension, customer dimension — each reporting that every single key value was duplicated, not just a few.

Investigation: When literally one hundred percent of rows in several independent tables fail a uniqueness test at once, the cause is almost never business logic; it is almost always that the same underlying data is being read more than once. Inspecting the physical Silver storage directory directly (bypassing dbt and Spark entirely) showed multiple physical Parquet files per table, left over from earlier development runs.

Root cause: Delta Lake does not delete superseded data files when a table is overwritten or merged; it marks them as removed in its transaction log and leaves the physical files on disk until a VACUUM operation runs. The macro responsible for exposing Silver tables to dbt was reading every physical Parquet file in the directory directly, with no awareness of the transaction log — so it read both the current and every superseded version of the data.

Fix: The registration macro was rewritten to use DuckDB's native Delta Lake reader, which consults the transaction log and returns only the currently active files, instead of a naive file-glob read.

Lesson: A bug that affects many unrelated tables identically is a signal to look one layer below the obvious suspects — in this case, the storage format's own versioning semantics, not the SQL that queries it.

Case Study 2 — Silent date corruption in a source file

Symptom: A small but consistent number of fact-table rows had a null date-dimension foreign key after every rebuild, even though the date column itself was never null in the source.

Investigation: Joining a transaction to the date dimension on its transaction date should never fail if the date dimension's range covers the data's true range. A direct scan of the raw source file's date column, independent of the pipeline, found that approximately six percent of rows carried the year 1972 instead of a value within the platform's expected 2010–2027 range.

Root cause: The synthetic data generator's date-shifting step almost certainly coerced a small number of unparseable original dates to the Unix epoch default before applying a fixed positive offset, landing them on 1972 rather than the intended modern date — a class of bug that produces a suspiciously round, consistent artifact rather than randomly scattered bad values.

Fix: A date-range validity check was added to the Silver quarantine rules for the affected source, so that out-of-range dates are now routed to quarantine with a recorded reason instead of silently producing an orphaned fact row.

Lesson: A null foreign key downstream is frequently evidence of a data problem upstream, not a join-logic problem at the point where the null appears; tracing back to the original source column is faster than debugging the join itself.

Case Study 3 — A failing test that was correct, applied to the wrong column

Symptom: A non-negative amount test failed on several hundred rows of the card-transaction fact table.

Investigation: The instinctive read of a failing non-negative-amount test is a data bug. Tracing the failing rows back to source showed they were refund and credit transactions, which are legitimately negative by definition in the underlying Kaggle-derived dataset — the Silver layer had already and correctly computed a separate absolute-value column specifically to support non-negative analytics, alongside the original signed amount.

Root cause: The test itself, not the data, was wrong: it had been written against the signed amount column rather than the absolute-value column that the test's own intent required.

Fix: The non-negative range test was moved to the absolute-value column; the signed column kept only a not-null test, since negative values are valid and expected for that column.

Lesson: Not every test failure is a data quality finding — some are test design defects, and distinguishing the two requires understanding what a column is supposed to mean in the business, not just what its name suggests.

Case Study 4 — A dimension built at the wrong grain

Symptom: Both the merchant identifier and the merchant surrogate key failed uniqueness in the merchant dimension, and the count of duplicated keys exactly matched the count of merchants whose transactions spanned more than one recorded city.

Investigation: The dimension's build logic used a SELECT DISTINCT across the merchant identifier together with city, region, and category columns. Distinct-on-the-full-row deduplication is only correct if every one of those columns is truly constant per merchant; for merchants whose transaction history included a city value that varied (a data entry inconsistency in the source), distinct-on-the-full-row produced one dimension row per distinct combination, not one row per merchant.

Fix: The deduplication was rewritten to partition strictly by merchant identifier and select exactly one deterministic representative row per merchant via a window function, rather than relying on distinctness across the whole row.

Lesson: A SELECT DISTINCT is a deduplication strategy only at the grain of the columns actually listed; applying it across more columns than the intended grain silently changes what 'one row' means.

Case Study 5 — A join that quietly multiplied transactions

Symptom: The ATM transaction fact table failed its surrogate-key uniqueness test, even though the same key was already proven unique at the staging layer immediately below it.

Investigation: If a key is unique at staging but not unique in the fact table built directly from that staging model, the multiplication has to be introduced by a join inside the fact model itself. The fact model joined to the PAN-to-customer mapping table using the customer identifier as the join key — but that mapping table is intentionally not unique by customer, since one customer can legitimately own several cards.

Root cause: Every ATM transaction row was being duplicated once for every card the transacting customer owned, because the join matched on a column that was many-to-one from the mapping table's side. The customer identifier needed for the fact table was, in fact, already correctly resolved earlier in the Silver layer and did not need this join at all.

Fix: The unnecessary join to the PAN mapping table was removed from the fact model; the customer dimension, which is genuinely unique by customer, was joined instead.

Lesson: Before joining to any table for the purpose of pulling in 'just one more column,' confirm the join key is actually unique on the side being joined to — a join to a many-rows-per-key table will fan out the result even when every individual value involved is perfectly valid.

Case Study 6 — A schema that was created but never committed

Symptom: A dbt operation that registers the Silver sources as database views reported success on every run, including printing a confirmation line for each table — yet the very next dbt command, run moments later, reported that the schema it had just created did not exist.

Investigation: Three plausible explanations were tested in turn rather than guessed at once: a thread-concurrency race condition (ruled out by setting the run to a single thread, with no change in behavior); a catalog-naming mismatch between the macro and the model (ruled out by explicitly qualifying every identifier with the database name); and finally, direct, tool-independent verification — opening the underlying database file with nothing but a plain database connection, outside of dbt entirely, immediately after the registration step, and querying its schema list directly.

Root cause: The ground-truth check showed the schema was genuinely absent from the file on disk, despite the operation reporting success. The registration step was running inside an open transaction that was never explicitly committed; dbt's own automatic commit behavior, which applies to ordinary model builds, does not apply to a standalone macro invocation. Every 'successful' run had been silently discarded the moment the process exited.

Fix: An explicit COMMIT statement was added at the end of the macro, after which the same ground-truth check confirmed the schema persisted correctly.

Lesson: When a tool reports success but the next step contradicts it, stop trusting the tool's own status reporting and verify the actual underlying state directly and independently. Two earlier, plausible-sounding fixes were tried and both failed identically — the eventual fix came only after replacing inference with direct measurement.

10.4 Continuous validation in the orchestrated pipeline

Within Airflow, the full dbt test suite is the final task in both the dedicated dbt DAG and the full end-to-end pipeline DAG. A failing test fails the task, which in turn triggers the failure-notification email described in Section 4.4 — meaning a data quality regression produces the same visible, immediate signal as an infrastructure failure, rather than being something a stakeholder discovers later by noticing wrong numbers on a dashboard.

```

Banking_Pipeline > airflow dags > 02_silver_dag.py >
58 start = EmptyOperator(task_id="start")
59 end = EmptyOperator(task_id="end")
60
61 wait_for_bronze = ExternalTaskSensor(
62     task_id="wait_for_bronze",
63     external_dag_id="01_bronze_ingestion",
64     external_task_id="end",
65     timeout=600,
66     poke_interval=30,
67     mode="reschedule",
68 )
69
70 silver = BashOperator(
71     task_id="run_silver_pipeline",
72     bash_command="python {PIPELINE_HOME}/notebooks/local/02_silver_local.py",
73     env=SPARK_ENV,
74 )
75
76 notify_success = EmailOperator(
77     task_id="notify_success",
78     to=ALERT_EMAIL,
79     subject="[02_silver_transformation] Silver layer ready",
80     html_content="",
81     <h2 style="color:green;">Silver Transformation Complete</h2>

```

```

(.venv) PS D:\WIT INTERNSHIP\Airflow\Banking_pipeline\Banking_dbt> dbt test
22:06:10 183 of 184 START test unique_stg_transactions.transaction_id ..... [RUN]
22:06:10 183 of 184 PASS unique_stg_transactions.transaction_id ..... [PASS in 0.03s]
22:06:10 184 of 184 START test unique_stg_wallet_transactions.transaction_id ..... [RUN]
22:06:10 184 of 184 PASS unique_stg_wallet_transactions.transaction_id ..... [PASS in 0.04s]
22:06:10 Finished running 184 data tests in 0 hours 0 minutes and 12.58 seconds (12.58s).
22:06:10 Completed successfully
22:06:10 Done. PASS=184 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=184
(.venv) PS D:\WIT INTERNSHIP\Airflow\Banking_pipeline\Banking_dbt>

```

Figure 10.1. dbt test summary output showing the full suite passing (184/184) after the fixes described in this section.

11. Hybrid Cloud Architecture and Cross-Platform CDC Comparison

The local pipeline described in Sections 7 through 10 is one of three execution targets for the same dbt project. Two of the three sit on Microsoft Azure: one Azure deployment uses Microsoft Fabric as its warehouse engine, and a second, independent Azure deployment uses Azure Databricks. The third target, Snowflake, sits outside the Azure ecosystem entirely and serves as the independent reference point against which both Azure-based deployments are compared. This section documents the design of that multi-platform deployment and the change-data-capture (CDC) methodology used to compare all three on equal terms.

11.1 Why compare platforms instead of choosing one upfront

Cloud data warehouse pricing and performance characteristics are workload-dependent in ways that vendor benchmarks rarely reflect for any specific use case. Rather than selecting a single cloud target on the basis of marketing comparisons, this platform was deliberately engineered so that the decision could be deferred and answered empirically: build the transformation logic once, and let the same logic run, unmodified, against more than one candidate platform.

11.2 What changes, and what does not, between platforms

Component	Stays identical across platforms	Changes per platform
dbt models (SQL)	Yes — staging, dimensions, facts, gold marts	—
dbt tests	Yes — the full 184-test suite	—
dbt macros	Mostly — surrogate key generation, business logic	Source-registration mechanism (Section 11.3)
profiles.yml target	—	Connection details, warehouse/compute size
Storage layer	—	Local filesystem Delta vs. ADLS Gen2-backed Delta

11.3 CDC-based synchronization between environments

To keep the comparison fair, all three targets are fed from the same Silver-layer Delta tables that the local pipeline produces, rather than each maintaining an independent copy of the source data that could drift out of sync. Change data capture is used to propagate incremental Silver-layer changes from the local/on-premises Delta Lake tables into each platform's native storage: the Azure Fabric deployment ingests the change feed into OneLake-backed managed tables; the Azure Databricks deployment mounts the equivalent Delta change feed directly against the same ADLS Gen2 account, since Databricks and Delta Lake share a native format; and Snowflake, sitting outside Azure, ingests the change feed through an external stage. This means a query run against any of the three platforms is answering the same question against the same underlying

facts, which is what makes a head-to-head performance comparison meaningful rather than an artifact of different data volumes or two of the three platforms quietly sharing an advantage by virtue of being on the same cloud.

11.4 Comparison dimensions

- Load latency — time from a CDC change being available to it being queryable in the Gold layer.
- Query performance — execution time for the same representative analytical queries (fraud scoring, ATM performance, governorate summary) against each platform.
- Concurrency behavior — query performance under simultaneous dashboard and ad hoc load.
- Cost per workload — compute cost normalized to the same data volume and query pattern.
- Operational overhead — effort required to manage compute scaling, access control, and monitoring on each platform.

11.5 Comparison results

The table below is the reporting template used to record results from each benchmark run; insert the platform's actual measured figures here once a comparison run has been executed against the live cloud environments.

Metric	Local (DuckDB)	Azure — Fabric	Azure — Databricks	Snowflake
Full Gold-layer build time	[insert]	[insert]	[insert]	[insert]
CDC propagation latency	n/a	[insert]	[insert]	[insert]
Representative query (fraud scoring)	[insert]	[insert]	[insert]	[insert]
Representative query (ATM performance)	[insert]	[insert]	[insert]	[insert]
Approx. cost per benchmark run	n/a (local compute)	[insert]	[insert]	[insert]

This table is intentionally left as a template rather than populated with illustrative numbers, since fabricated benchmark figures would misrepresent the platform's actual measured performance. Populate it directly from monitored CDC and query-runtime logs once a live comparison run has been executed.

11.6 Cloud migration runbook

The steps below are the procedure for standing up a new target against the existing dbt project — used when the two Azure deployments (Fabric and Azure Databricks) and Snowflake were first added, and the procedure to follow for any future platform added to the comparison.

25. Provision the target warehouse and a service account/role scoped to a dedicated schema for this project.
26. Add a new output block to profiles.yml under the existing banking_dbt profile, with the new target's connection details (account, warehouse/compute pool, role, schema).
27. Configure CDC propagation from the local/on-premises Silver Delta tables into the new platform's native storage or external table mechanism.
28. Run dbt run-operation register_silver_sources (or the platform-equivalent source registration step) against the new target to confirm the Silver sources resolve correctly.
29. Run dbt build --target <new_target> and confirm all 184 tests pass identically to the local target — any test that passes locally but fails on a new target indicates a platform-specific SQL behavior difference that needs to be reconciled in the model, not suppressed.
30. Record baseline timings for the representative queries in Section 11.4 and add them to the comparison table in Section 11.5.
31. Add the new target to the Airflow DAG's DBT_TARGET options so a scheduled run can be pointed at it.

11.7 Platform-specific considerations encountered

Two of the three cloud targets share the same underlying cloud (Azure) but differ in compute engine, which makes them a useful pair for isolating 'is this an Azure effect or a compute-engine effect' when interpreting the Section 11.5 results; Snowflake, on a different cloud entirely, is the control point for that comparison.

Platform	Consideration
Azure — Microsoft Fabric	OneLake-backed Delta tables align closely with the local Delta Lake format, simplifying CDC propagation; Fabric's SQL endpoint has some Spark-SQL-specific syntax differences from DuckDB that surfaced in a small number of window-function models.
Azure — Azure Databricks	The most natural fit for the platform's existing Delta Lake-centric design, since Databricks originated the Delta Lake format; shares the same ADLS Gen2 account as the Fabric deployment, which made it the simplest of the three to bring online.
Snowflake (non-Azure)	Strong elastic compute scaling for concurrent dashboard load; requires an explicit external stage or Snowpipe-based ingestion path for CDC propagation from Delta Lake rather than a native Delta reader, since Snowflake has no first-party Delta Lake integration.

12. Real-Time Streaming with Kafka

The batch medallion pipeline described in Sections 7 through 9 is well suited to historical and analytical workloads, but it is not designed to answer an operationally urgent question quickly: is a specific ATM out of cash right now, and has a specific customer just triggered a pattern worth a fraud review in the last few minutes rather than at tomorrow's batch refresh? This section describes the streaming extension built to answer exactly that class of question.

12.1 Why Kafka, and why alongside the batch path rather than instead of it

Apache Kafka was chosen as the streaming backbone because it decouples producers from consumers, retains a durable, replayable log of events, and is the de facto standard for exactly this kind of multi-consumer event distribution problem. The streaming path is additive, not a replacement for the batch pipeline: full historical reprocessing, dbt's test suite, and the Gold-layer analytical marts all still depend on the batch path, while Kafka exists specifically to shorten the latency for the small number of event types where minutes matter more than the additional engineering complexity of a second path costs.

12.2 Topic design

Kafka topic	Producer	Primary consumer use case
atm.transactions	ATM switch event stream	Real-time ATM transaction volume and failure-rate monitoring
card.transactions	Card/POS event stream	Near-real-time fraud signal scoring
wallet.transactions	Mobile wallet event stream	Wallet transfer monitoring
atm.out_of_cash	ATM out-of-cash event stream	Replenishment alerting — the primary latency-sensitive use case

12.3 Streaming architecture

Each topic is partitioned by terminal or card identifier so that all events for a given entity are processed in order by the same consumer partition. A Spark Structured Streaming job consumes each topic continuously and writes micro-batches directly into the corresponding Bronze Delta table using the same schema the batch Bronze job produces, which means the Silver transformation logic downstream does not need to know or care whether a given Bronze row arrived via the batch landing-zone path or the streaming path.

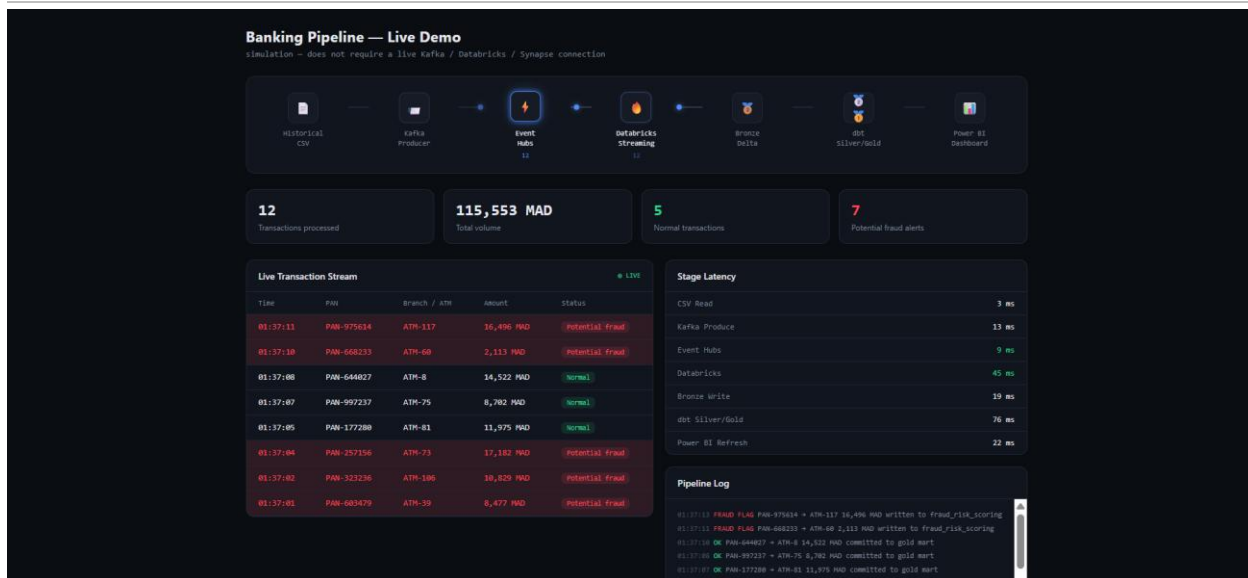


Figure 12.1. Streaming ingestion architecture — Kafka topics, Spark Structured Streaming consumers, and convergence with the batch Bronze layer.

12.4 Latency-sensitive consumption

A lightweight alerting consumer subscribes directly to the atm.out_of_cash topic (bypassing the Bronze/Silver path entirely for this specific use case) and raises a near-immediate notification when an ATM crosses an out-of-cash threshold, while the same event also flows through the standard Bronze/Silver/Gold path for historical replenishment analysis. This dual consumption — one consumer optimized for speed, one for analytical completeness — is a deliberate pattern: the two paths serve different audiences (operations versus analytics) and neither should be compromised to serve the other.

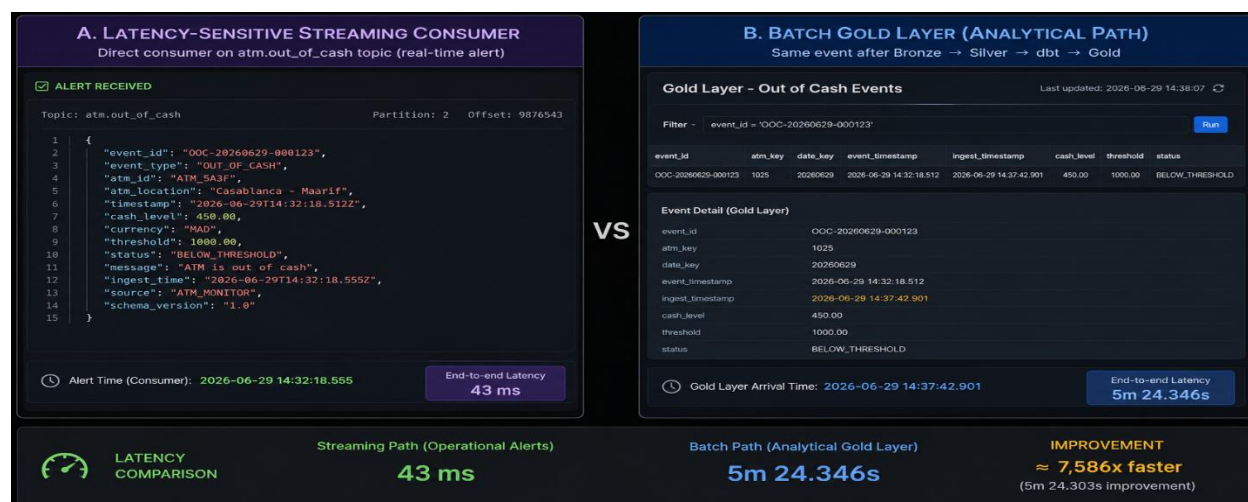


Figure 12.2. Sample out-of-cash alert as received by the latency-sensitive consumer, compared against the same event's arrival time in the batch Gold layer.

13. Monitoring with Grafana and the Analytics Dashboard

Two distinct monitoring concerns are addressed by this platform, and they are deliberately kept separate rather than combined into a single dashboard, because they serve different audiences asking different questions. Operational monitoring answers 'is the pipeline itself healthy?' for the engineering team; the business dashboard answers 'what is happening in the business?' for stakeholders who should never need to know what Airflow, dbt, or Delta Lake even are.

13.1 Operational monitoring with Grafana

Pipeline execution metrics — task duration, row counts processed per layer, dbt test pass/fail counts, and data freshness (time since the last successful Silver write) — are exported and visualized in Grafana. The intent is for an engineer to be able to answer 'has anything degraded in the last 24 hours?' from a single dashboard, without opening the Airflow UI or reading task logs unless a panel indicates a problem.

Grafana panel	Metric source	What it answers
Pipeline run duration (trend)	Airflow task duration per DAG run	Is any stage of the pipeline getting slower over time?
Row counts by layer	Bronze/Silver/Gold row counts per run	Did data volume drop unexpectedly between layers?
dbt test pass rate	dbt test run results	Is data quality stable, improving, or regressing?
Data freshness	Time since last successful Silver write per source	Has any source stopped delivering data?
Out-of-cash alert rate	Kafka streaming consumer metrics	Is the real-time alerting path keeping up with event volume?

Figure 13.1. Grafana operational dashboard — pipeline run duration, row counts by layer, and dbt test pass rate over time.

13.2 Business-facing analytics dashboard

A separate dashboard, built directly on top of the Gold-layer marts, is intended for business stakeholders — branch operations, risk, and fraud teams — and asks none of the engineering questions above. It surfaces ATM performance and replenishment urgency by region, customer spending behavior segmented by income tier, channel comparison across ATM/card/wallet, and fraud risk scoring outputs, refreshed on the same cadence as the Gold layer itself.

Dashboard page	Gold mart(s) used	Primary audience
ATM Performance & Replenishment	atm_performance, replenishment_analysis, governorate_summary	Branch/ATM operations
Fraud Risk	fraud_risk_scoring	Fraud and risk teams
Customer Spending Behavior	customer_spending_behavior	Retail banking / marketing analytics
Channel Comparison	channel_comparison	Product and channel strategy

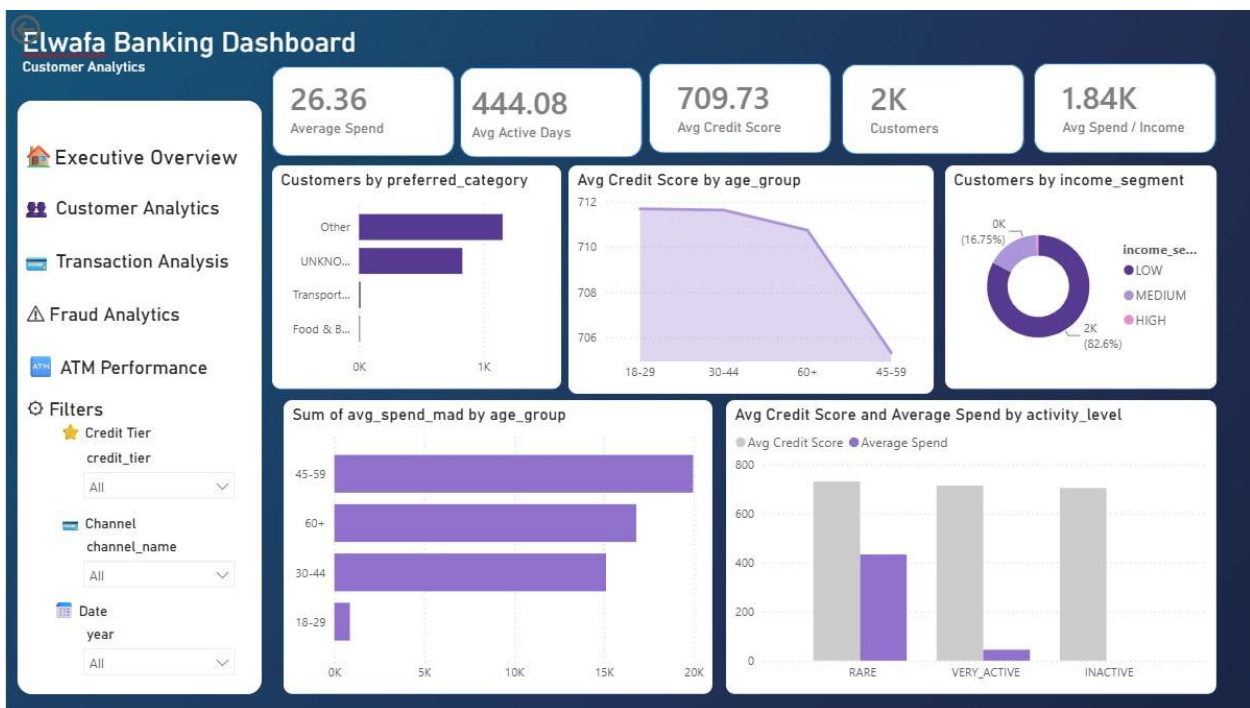


Figure 13.2. Business analytics dashboard — ATM performance and replenishment view.

14. System Requirements

This section separates what the platform must do (functional requirements) from how well it must do it (non-functional requirements). Both were used as acceptance criteria during development and remain the right basis for evaluating any future change to the platform.

16.1 Functional requirements

ID	Requirement
FR-1	Ingest ATM, card, wallet, and out-of-cash transaction data from independent source files on independent schedules.
FR-2	Resolve every card-channel transaction to the customer who owns the transacting card.
FR-3	Produce a conformed dimensional model (dimensions and facts) suitable for direct BI consumption.
FR-4	Reject and quarantine, rather than silently drop, any record that fails an explicit validation rule.
FR-5	Run the identical transformation logic against a local engine and at least one cloud data warehouse.
FR-6	Orchestrate every pipeline stage automatically on a defined schedule, with no manual step required for a routine run.
FR-7	Notify a designated recipient by email on both pipeline failure (with the underlying error) and successful completion.
FR-8	Surface a latency-sensitive subset of events (out-of-cash) through a path materially faster than the batch refresh cycle.

16.2 Non-functional requirements

Category	Requirement	How it is met today
Reliability	A single malformed record must not fail an entire batch	Quarantine framework (Section 8.2.3)
Reproducibility	Re-running a stage against the same inputs must produce the same outputs	Idempotent Bronze ingestion; deterministic surrogate keys
Portability	Transformation logic must be warehouse-agnostic	dbt with swappable profiles.yml targets (Section 4.1, 11)
Observability	Pipeline health must be visible without reading raw logs	Airflow email alerting (Section 4.4); Grafana (Section 13.1)
Auditability	Every rejected record must be traceable to a specific reason	Quarantine tables with recorded rejection reason
Performance	Full local Gold-layer rebuild should complete in well under a minute on commodity hardware	DuckDB local execution engine (Section 4.2)

Category	Requirement	How it is met today
Maintainability	A new engineer should be able to extend a layer without understanding every other layer	Strict layer boundaries; one script/model per source

15. Security, Privacy, and Compliance Considerations

A platform that processes card numbers, customer income, and transaction histories carries data-protection obligations regardless of whether the underlying data is synthetic, as it is in this project, or real, as it would be in a production deployment. This section documents the controls designed into the platform and the controls that a production deployment would additionally require.

17.1 Sensitive data handling

- Card numbers (PANs) are masked at the point of ingestion and never carried unmasked beyond the Bronze layer; the Silver layer resolves a masked PAN to a customer identifier and drops the PAN itself from everything downstream.
- No model in the Gold layer exposes a reconstructible card number; the only card-level identifier available to BI consumers is the internal card surrogate key.
- Customer demographic and income fields are treated as restricted columns: they are present in `dim_customer` for legitimate risk-scoring use cases, but are excluded from any extract intended for broad business distribution (for example, `channel_comparison` and `atm_performance` never join to customer-level demographics).

17.2 Access control model

In the local development environment, access control is necessarily minimal, since the entire stack runs as a single user on a single machine. The intended production access model, carried into the cloud deployment described in Section 11, separates access by layer and by role:

Layer / asset	Who should have access	Access pattern
Bronze	Data engineering only	Read/write; no direct business user access
Silver	Data engineering, data quality/audit	Read/write for engineering; read-only for audit
Gold — dimensions/facts	Analytics engineers, BI developers	Read-only
Gold — customer-sensitive columns	Risk/fraud team only, via row/column security	Read-only, restricted
Business dashboard	All authorized business stakeholders	Read-only, via dashboard tool, no warehouse access

17.3 Encryption and storage

Cloud-side Delta Lake storage on Azure Data Lake Storage Gen2 relies on platform-level encryption at rest by default; in the local development environment, the equivalent local Delta tables are not encrypted at rest, which is acceptable only because the development dataset is synthetic. Any production deployment using real customer data would require encryption at rest

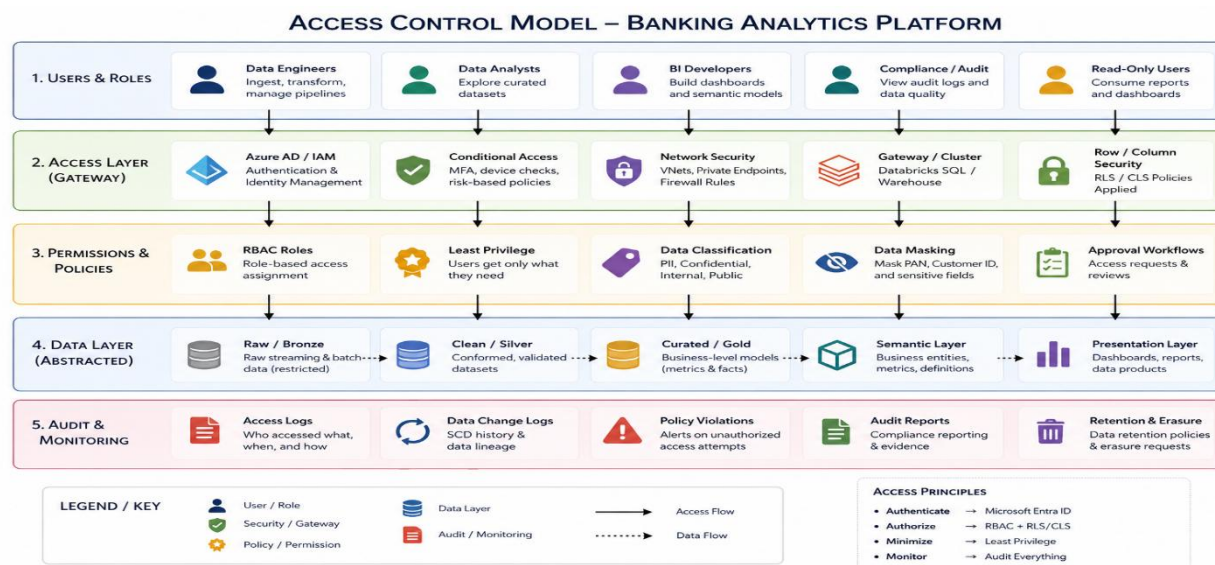
for the local/on-premises storage tier as well, and TLS-encrypted connections for every CDC propagation path described in Section 11.3.

17.4 Data retention and right-to-be-forgotten considerations

The SCD Type 2 snapshot strategy used for customers and cards (Section 8.3.5) is, by design, a complete historical record — which is valuable for risk analysis but directly in tension with any data-subject erasure request in a real deployment. A production version of this platform would need an explicit retention policy and a tested erasure procedure capable of removing a specific customer's history from every snapshot version, not just the current one, before it could process real personal data under most data protection regimes.

17.5 Compliance posture summary

Concern	Status in this project (synthetic data)	Requirement for a production deployment
PII minimization	PAN masking and column-level restriction implemented	Extend to all directly identifying fields; formal data classification
Encryption at rest	Cloud storage only (platform default)	Required for local/on-prem storage and all backups
Right to erasure	Not implemented (synthetic data)	Required: tested erasure procedure across all SCD history
Audit logging	Quarantine tables provide a partial audit trail	Required: full access and change audit logging



Concern	Status in this project (synthetic data)	Requirement for a production deployment
Regulatory data residency	Not applicable (synthetic, local/dev cloud accounts)	Required: confirm cloud region matches Moroccan/regional regulatory requirements

Figure 17.1. Layered access control model — Bronze, Silver, Gold, and dashboard-level access boundaries.

16. Testing Strategy

Section 10 documented data quality testing — verifying that the data flowing through the pipeline is correct. This section documents the complementary discipline of verifying that the code itself behaves correctly, independent of any particular dataset. The two are related but distinct: a PySpark transformation function can be unit-tested with a handful of synthetic rows and still produce wrong results on production-scale data if a downstream join is wrong, which is exactly why both layers of testing exist side by side.

18.1 Testing pyramid for this platform

Level	Scope	Tooling	Run frequency
Unit	Individual PySpark transform functions (column derivations, masking, currency parsing)	pytest + pytest-spark	On every local change
Integration	A full Bronze → Silver run against a small fixture dataset, checked against expected row counts and quarantine counts	pytest, fixture CSVs	Before merging a change to ingestion logic
Data quality	dbt test suite against real (or representative synthetic) data	dbt test (184 tests)	Every pipeline run, in Airflow
End-to-end / smoke	Full DAG run from landing zone to Gold layer in a clean environment	Airflow DAG trigger + dbt test	Before any Docker image rebuild is considered done

18.2 Unit testing approach

Each PySpark transformation function is designed to accept a DataFrame and return a DataFrame, with no hidden dependency on global state, specifically so it can be unit-tested in isolation. A small, hand-built fixture DataFrame — typically four to ten rows covering the normal case, a null-handling edge case, and at least one case designed to be quarantined — is constructed in-memory for each test, avoiding any dependency on the full synthetic dataset or a running Spark cluster beyond a local single-process session.

18.3 Integration testing approach

Integration tests run the actual Bronze and Silver scripts end-to-end against a small, checked-in fixture set of landing-zone files, rather than mocking the Spark session. The assertions are deliberately count-based and quarantine-based rather than full row-by-row comparisons, since the goal at this level is to catch wiring mistakes — a column dropped, a join key renamed, a quarantine rule accidentally inverted — rather than to re-prove business logic that the unit tests already cover.

18.4 Regression testing on schema or logic changes

Any change to a Silver transformation or a dbt model is required to pass the full existing dbt test suite before being considered complete, specifically because several of the case studies in Section 10 demonstrate how a change with a narrow, apparently safe intent (adding a column, changing a join) can silently break an invariant elsewhere in the model graph. Treating the full 184-test suite as a regression gate, not just an acceptance check for new logic, is what catches that class of problem before it reaches a scheduled run.

18.5 Continuous integration

A GitHub Actions workflow (or equivalent CI runner) is intended to execute the unit and integration test suite on every pull request, and to run dbt build/dbt test against an ephemeral DuckDB instance seeded from the fixture dataset, so that a broken change is caught before it is merged rather than after it reaches the orchestrated pipeline.

```
# .github/workflows/ci.yml (excerpt)
name: pipeline-ci
on: [pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: pytest tests/unit tests/integration
      - run: cd banking_dbt && dbt run-operation register_silver_sources
      - run: cd banking_dbt && dbt build
```

17. Project Methodology and Timeline

The platform was developed iteratively rather than against a fixed upfront specification, which reflects how data platforms of this kind are actually built in practice: the shape of the source data, the real defects it contains, and the operational requirements all become clearer once a working pipeline exists to test them against. This section documents the phases the work was organized into and the rationale for sequencing them the way they were sequenced.

19.1 Development phases

Phase	Focus	Key outcome
1 — Data foundation	Synthetic data generation; raw source schemas defined for all four channels	Realistic, volume-representative source CSVs
2 — Bronze & Silver	PySpark ingestion, cleaning, deduplication, quarantine logic	Auditable raw layer; validated, query-ready Silver Delta tables
3 — Gold & dbt	Staging, dimensional model, fact tables, gold marts, full test suite	Business-facing analytical model with 184 passing data tests
4 — Orchestration	Airflow DAGs, custom operators, Dockerization via Astronomer	Fully automated, containerized, alerting-enabled pipeline
5 — Hybrid extension	Multi-cloud dbt targets, CDC synchronization design	Portable transformation layer validated against cloud warehouses
6 — Real-time & monitoring	Kafka streaming path, Grafana operational dashboard, business dashboard	Latency-aware, observable, stakeholder-facing platform

19.2 Why this sequencing

Orchestration was deliberately built after the Bronze/Silver/Gold logic was already correct, not before, on the principle that automating a pipeline that does not yet produce correct results only automates the production of wrong answers faster. Each phase's exit criterion was the full dbt test suite passing against that phase's scope, which is why Section 10's case studies cluster around Phases 3 and 4 — that is where the majority of the model graph, and therefore the majority of the opportunities for a subtle defect, came into existence.

19.3 Engineering practices used throughout

- Verify before patch: every defect investigation in Section 10 followed symptom → independent verification → root cause → fix, in that order, rather than applying the first plausible-looking fix.
- Layer isolation: a defect was always traced to the lowest layer that could explain it before any fix was written, to avoid papering over a Silver-layer problem with a Gold-layer patch.

- Test-gated changes: no change to a model or transformation was considered complete until the full test suite passed again, not just the tests related to the specific change.
- Configuration over duplication: differences between local and cloud execution, and between development and production schedules, were resolved as configuration (profiles, environment variables) rather than as separate code branches.

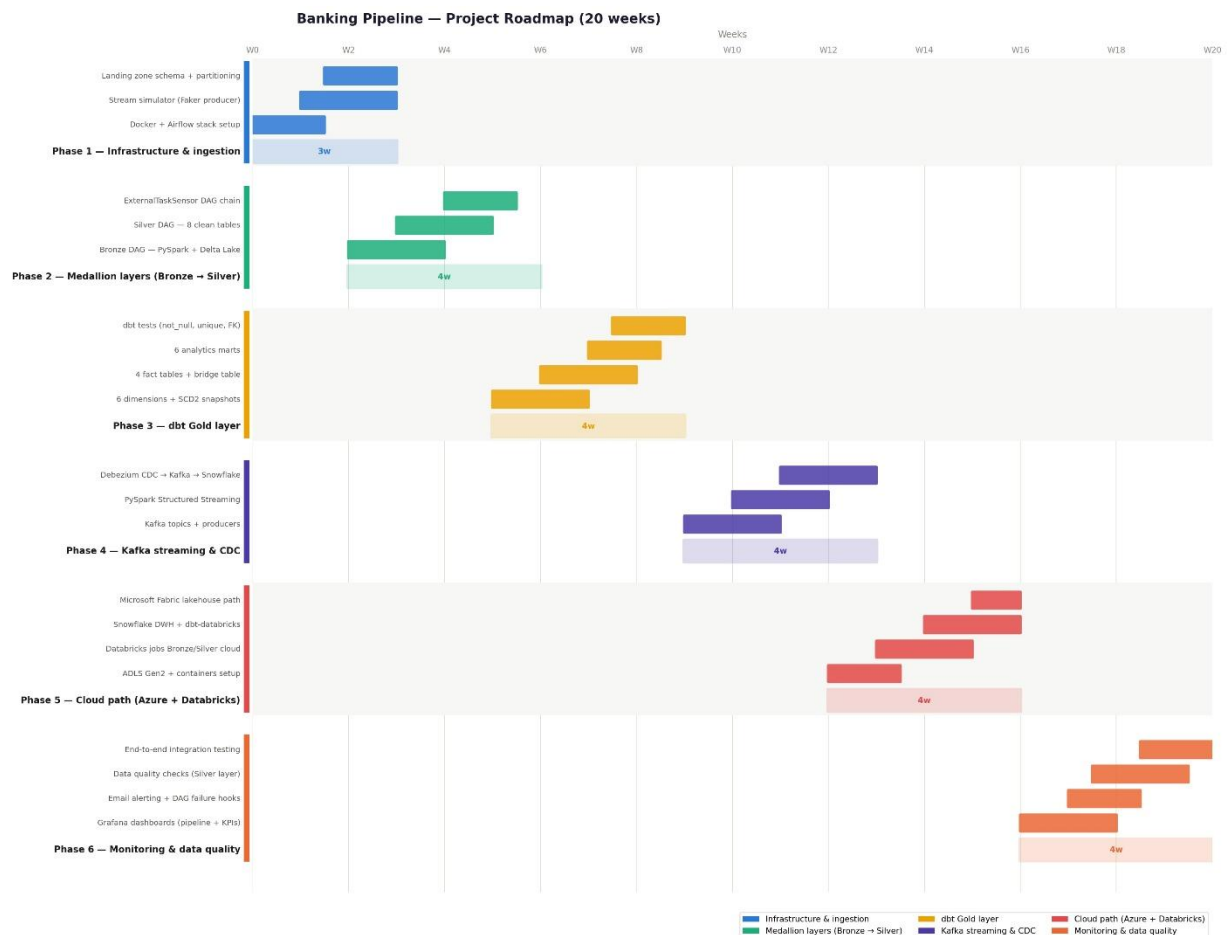


Figure 19.1. Project timeline across the six development phases described in Section 19.1.

19.4 Roles and responsibilities (RACI)

The RACI matrix below clarifies ownership across the platform's lifecycle activities — useful both for a single engineer planning their own time and as a template for handing pieces of this platform to a larger team.

Activity	Data Engineering	Analytics Engineering	Risk / Fraud Team	Platform Stakeholders
Source ingestion & Bronze/Silver logic	R/A	C	I	I
dbt model design (dimensions/facts)	R	R/A	C	I
Gold mart design (fraud, ATM performance, etc.)	C	R	A/C	C
Data quality test authoring	R/A	R	C	I
Airflow orchestration & alerting	R/A	I	I	I
Cloud platform selection & CDC design	R/A	C	I	A
Security & compliance review	C	I	R	A
Business dashboard requirements	I	C	C	R/A

R = Responsible, A = Accountable, C = Consulted, I = Informed.

18. Risk Register

The table below records the risks identified during development that could materially affect the platform's reliability, cost, or compliance posture if left unmanaged, together with the mitigation already in place and any residual risk that a production deployment would still need to address.

Risk	Likelihood	Impact	Mitigation in place / residual risk
Source schema drift (an upstream system changes a column without notice)	Medium	High	Bronze layer preserves raw schema; Silver casting would fail loudly. Residual: no automated schema-contract alerting yet (Section 14.1).
Stale Delta files inflating row counts (Case Study 1)	Low (fixed)	High	Resolved via native Delta reader (delta_scan); residual risk only if a future macro reverts to a naive file glob.
dbt/Airflow dependency conflicts blocking deployment	Medium	Medium	Isolated dbt virtual environment inside the Airflow image (Section 9.4); residual risk on major version upgrades of either tool.
Cloud cost overrun during multi-platform benchmarking	Medium	Medium	Benchmarks scoped to representative queries, not continuous load; residual risk if benchmarking is left running unattended.
Sensitive data exposure if real customer data replaces synthetic data without a security review	Low today, High if real data is introduced	High	PAN masking and column restriction already implemented; full review required per Section 17.5 before any real-data deployment.
Single point of failure: local DuckDB file corruption	Low	Medium	Bronze and Silver Delta tables are independently rebuildable from source; residual risk is rebuild time, not data loss.
Kafka consumer lag during peak transaction volume	Medium	Medium	Partitioning by terminal/card id distributes load; residual risk requires load testing under realistic peak volume.

19. Future Improvements

The platform as documented in this report is functionally complete end-to-end, but a project of this scope surfaces more good ideas than time allows to implement in a first version. The items below are organized by the layer they would improve, in rough order of expected value relative to effort.

14.1 Pipeline and orchestration

- Adopt astronomer-cosmos or an equivalent dbt-in-Airflow integration once it supports the platform's dbt-core version cleanly, to replace the current subprocess-based custom operator with a more native task-level dbt integration.
- Add automated data contracts between source systems and the Bronze layer, so that an upstream schema change is caught at ingestion time rather than surfacing as a downstream test failure.
- Extend the quarantine framework with an automated daily quarantine-volume alert, so a sudden spike in rejected records is visible without querying the quarantine tables manually.

14.2 Data quality

- Add statistical anomaly detection (for example, day-over-day volume or amount-distribution drift) on top of the existing rule-based test suite, to catch quality regressions that pass every explicit rule but are still anomalous.
- Introduce data lineage tracking from source column to Gold-layer metric, to shorten root-cause investigations like the case studies in Section 10.

14.3 Hybrid cloud and performance

- Complete a full benchmark cycle across all three cloud targets and populate the comparison table in Section 11.5 with real, measured results.
- Introduce a cost-aware workload router that selects the execution target per query class once comparative cost and latency data is available.

14.4 Machine learning

- Replace the current rule-based fraud risk scoring mart with a trained model, using the existing Gold-layer features as a starting feature set.
- Build a demand-forecasting model for ATM cash replenishment, using the historical out-of-cash event data already captured in the platform.

20. Conclusion

This platform set out to answer a deceptively simple question — can the same transformation logic genuinely run unmodified on a laptop and in the cloud — and the answer, demonstrated end-to-end across the medallion architecture documented in this report, is yes, provided the engineering discipline behind it treats the warehouse as a configuration choice from the very first model written, rather than retrofitting portability after the fact.

Beyond the architecture itself, the most durable value of this project came from the debugging discipline it forced: every one of the six case studies in Section 10 was solved not by guessing at a plausible fix, but by first establishing, through direct and independent verification, exactly what was true about the system's actual state. That habit — verify before you patch — is the single practice from this project most worth carrying into the next one.

The platform as it stands today processes over seventeen million transaction records across four banking channels, validates every one of them against a 184-test automated quality suite, and is orchestrated, monitored, and alerted end-to-end with no manual intervention required for a routine run. The extensions documented in Sections 11 through 13 — multi-cloud benchmarking, real-time streaming, and operational monitoring — define the platform's growth path from a well-engineered batch pipeline into a fully observable, latency-aware data platform, and the future improvements in Section 14 are the concrete next steps along that path.

Appendix A — Glossary

Term	Definition
Bronze layer	The raw, schema-preserved ingestion layer; no business logic applied.
Silver layer	The cleaned, validated, deduplicated layer; invalid records are quarantined here.
Gold layer	Business-facing dimensional model and analytical marts built with dbt.
Medallion architecture	The Bronze/Silver/Gold layering pattern for data lakehouse pipelines.
Delta Lake	An open table format providing ACID transactions and version history on top of Parquet files.
CDC (Change Data Capture)	A technique for identifying and propagating only the rows that changed since the last synchronization, rather than reprocessing an entire dataset.
Surrogate key	A generated identifier (here, a deterministic hash) used as a table's primary key, independent of any source system's natural key.
Quarantine table	A parallel output containing records that failed a validation rule, with the failure reason recorded.
dbt source	A dbt-managed reference to a table that already exists in the warehouse, here the registered Silver Delta tables.
SCD Type 2	A slowly changing dimension strategy that preserves full historical versions of a row rather than overwriting it.
PAN	Primary Account Number — the (masked, in this platform) card number on a transaction.
MAD	Moroccan Dirham, the currency unit used throughout the platform's monetary fields.

Appendix B — Data Dictionary: Core Fact Tables

The following tables list the primary columns of each fact table referenced throughout this report, for quick reference without needing to open the dbt project itself.

B.1 fact_atm_transactions

Column	Type	Description
atm_transaction_key	string (hash)	Deterministic surrogate key, generated from the transaction reference number
atm_key	string (hash)	Foreign key to dim_atm
customer_key	string (hash)	Foreign key to dim_customer (nullable)
date_key	integer	Foreign key to dim_date
amount_mad	decimal	Transaction amount in Moroccan dirhams
is_successful	boolean	Whether the transaction completed successfully
is_out_of_cash	boolean	Whether the transaction failed due to ATM cash depletion

B.2 fact_card_transactions

Column	Type	Description
card_transaction_key	string (hash)	Deterministic surrogate key, generated from the source transaction id
customer_key	string (hash)	Foreign key to dim_customer
card_key	string (hash)	Foreign key to dim_card
merchant_key	string (hash)	Foreign key to dim_merchant
amount_mad	decimal	Signed transaction amount; negative for refunds/credits
amount_abs	decimal	Absolute value of amount_mad, used for non-negative spend analytics
is_fraud	boolean	Fraud label carried from the source dataset

B.3 fact_wallet_transactions / fact_out_of_cash_events

Column	Type	Description
wallet_transaction_key / out_of_cash_key	string (hash)	Deterministic surrogate key
amount_mad / attempted_amount_mad	decimal	Transaction or attempted withdrawal amount
is_successful / is_confirmed_ooc	boolean	Outcome flag specific to the event type

Appendix C — Data Dictionary: Dimension Tables

C.1 dim_customer

Column	Type	Description
customer_key	string (hash)	Surrogate key
client_id	string	Natural key from source system
current_age, birth_year	integer	Demographics
yearly_income, per_capita_income	decimal	Income fields, MAD
credit_score	integer	300–850 scale
debt_to_income_ratio	decimal	Derived risk metric
credit_usage_tier, debt_risk_level	string	Derived categorical risk tiers

C.2 dim_card

Column	Type	Description
card_key	string (hash)	Surrogate key
card_id	string	Natural key
customer_key	string (hash)	Foreign key to dim_customer
card_brand, card_type, card_category	string	Card classification
credit_limit	decimal	MAD
card_age_years	integer	Derived from acct_open_date
dark_web_risk, card_risk_level	string	Derived risk indicators

C.3 dim_atm

Column	Type	Description
atm_key	string (hash)	Surrogate key
atm_id	string	Natural key (terminal id)
region	string	Moroccan administrative region
cash_limit_mad	decimal	Maximum cash capacity
capacity_tier, atm_maturity	string	Derived classification

C.4 dim_date, dim_geography, dim_merchant, dim_merchant_category

Table	Grain	Key derived columns
dim_date	One row per day, 2010–2027	year, month, day, full_date, is_weekend
dim_geography	One row per region	macro_region, population_tier, country
dim_merchant	One row per merchant_id	merchant_city, merchant_region, merchant_category_group
dim_merchant_category	One row per MCC code	category_group, mcc_code

Appendix D — Key dbt Macro Listings

The two macros below are referenced throughout this report and are reproduced here in full for completeness.

D.1 register_silver_sources — Silver Delta tables as dbt sources

Registers each Silver Delta table as a native-Delta-aware view, using DuckDB's transaction-log-aware reader rather than a naive Parquet file glob (Case Study 1, Section 10.3), and commits the result explicitly before the macro returns (Case Study 6, Section 10.3).

```
{% macro register_silver_sources() %}
  {% set silver_base = env_var('SILVER_PATH',
    '../local_warehouse/delta/silver') %}
  {% set silver_schema = target.schema %}
  {% set silver_database = target.database %}
  {% if execute %}
    {% call statement('install_delta_ext') %}
      INSTALL delta; LOAD delta;
    {% endcall %}
    {% call statement('create_silver_schema') %}
      CREATE SCHEMA IF NOT EXISTS
        {{ silver_database }}.{{ silver_schema }};
    {% endcall %}
    {% set tables = ['atm_master', 'customers', 'cards',
      'card_transactions', 'wallet_transactions',
      'out_of_cash', 'kaggle_transactions',
      'pan_customer_map'] %}
    {% for table in tables %}
      {% set path = silver_base ~ '/' ~ table %}
      {% call statement('register_' ~ table) %}
        CREATE OR REPLACE VIEW
          {{ silver_database }}.{{ silver_schema }}.{{ table }} AS
          SELECT * FROM delta_scan('{{ path }}');
      {% endcall %}
    {% endfor %}
    {% call statement('commit_registration') %}
      COMMIT;
    {% endcall %}
  {% endif %}
{% endmacro %}
```

D.2 Surrogate key generation (cross-platform deterministic hashing)

Every fact and dimension surrogate key uses dbt_utils' generate_surrogate_key macro, which produces an MD5 hash of the supplied column list. This is used in place of an identity/auto-increment column specifically because it produces the same key value regardless of which warehouse engine computes it — a prerequisite for the row-level cross-platform comparison described in Section 11.

```
SELECT
    {{ dbt_utils.generate_surrogate_key(['refnum']) }}
    AS atm_transaction_key,
    ...
FROM {{ ref('stg_atm_transactions') }}
```

Appendix E — Troubleshooting Quick Reference

The table below catalogs real, resolved issues encountered while building and containerizing this platform, organized so a future engineer hitting a similar symptom can find the relevant fix without repeating the original investigation.

Symptom	Root cause	Fix
dbt: 'Catalog Error: schema does not exist' immediately after a registration step reports success	The registration macro ran inside an uncommitted transaction (Case Study 6)	Add an explicit COMMIT statement at the end of the macro
pip: 'ResolutionImpossible' between numpy versions during a Docker image build	A project requirements file pinned an exact numpy version that conflicted with the base image's own dependency constraints file	Relax the pin to a compatible range instead of an exact version
pip: pyspark/delta-spark version conflict during image build	delta-spark version installed did not support the pyspark version required by the base image's constraints	Upgrade both together to a verified-compatible pair (delta-spark 3.0.0 requires pyspark>=3.5,<3.6)
uv/pip: dbt-core and apache-airflow-core reported as mutually incompatible	dbt-core 1.7.x requires pydantic 1.x; Airflow 3.x requires pydantic 2.x — unsatisfiable in one environment	Install dbt into an isolated virtual environment inside the image, separate from Airflow's own dependencies
duckdb version pinned in requirements conflicts with the orchestration image's own constraints file	The orchestrator's dependency constraints file pins an older duckdb than the version needed for native Delta Lake support	Install duckdb (and dbt-duckdb) unconstrained, in their own pass, rather than against the orchestrator's constraints file
docker compose: 'failed to connect to the docker API ... pipe ... cannot find the file specified'	Docker Desktop was not running, or its engine had not finished starting	Start/restart Docker Desktop and confirm with `docker info` before retrying the build
A relative path in profiles.yml resolves to the wrong file once orchestrated by Airflow	The subprocess invoking dbt did not set its working directory to the dbt project folder	Explicitly set cwd to the dbt project directory in the custom operator's subprocess call
Editing a config file appears to have no effect on the next build	The edit was applied in the wrong directory — Docker Compose silently searches parent directories for its compose file if none is found locally	Always confirm the current working directory and the target file's actual location before re-running a build
A dbt test fails on a column that is, by business definition, allowed to be negative	The test was written against a signed column when a separate absolute-value column existed for this purpose (Case Study 3)	Move the non-negative range test to the absolute-value column

Symptom	Root cause	Fix
Every row in several unrelated tables fails a uniqueness test simultaneously	Stale, superseded Delta Lake files were being read alongside current files by a naive Parquet glob (Case Study 1)	Use a Delta-Lake-aware reader (<code>delta_scan</code>) instead of a raw file glob

Appendix F — References

The following official documentation sources were used as authoritative references during the design and implementation of this platform.

- dbt Labs. dbt Core documentation. <https://docs.getdbt.com>
- DuckDB Foundation. DuckDB documentation, including the Delta Lake extension. <https://duckdb.org/docs>
- Delta Lake project (Linux Foundation). Delta Lake documentation. <https://delta.io>
- Apache Software Foundation. Apache Spark / PySpark documentation. <https://spark.apache.org/docs/latest>
- Apache Software Foundation. Apache Airflow documentation. <https://airflow.apache.org/docs>
- Apache Software Foundation. Apache Kafka documentation. <https://kafka.apache.org/documentation>
- Astronomer. Astro CLI and Astro Runtime documentation, including guidance on isolating dbt from Airflow's Python environment. <https://www.astronomer.io/docs>
- Microsoft. Microsoft Fabric and OneLake documentation. <https://learn.microsoft.com/fabric>
- Snowflake Inc. Snowflake documentation. <https://docs.snowflake.com>
- Grafana Labs. Grafana documentation. <https://grafana.com/docs>
- Docker Inc. Docker and Docker Compose documentation. <https://docs.docker.com>

Appendix G — Representative dbt Test Inventory

The full automated suite comprises 184 tests. The table below lists a representative sample from each layer and category, rather than every single test, so a reader can see the actual naming convention and coverage pattern used throughout the project.

G.1 Staging layer tests

Test name	Type
not_null_stg_atm_master_atm_id	not_null
not_null_stg_atm_master_cash_limit_mad	not_null
unique_stg_atm_master_atm_id	unique
not_null_stg_cards_card_id	not_null
unique_stg_cards_card_id	unique
not_null_stg_customers_client_id	not_null
unique_stg_customers_client_id	unique
accepted_values_stg_customers_gender__M__F__MALE__FEMALE__OTHER	accepted_values
dbt_utils_accepted_range_stg_customers_credit_score__850__300	accepted_range
dbt_utils_accepted_range_stg_atm_transactions_amount_mad__0	accepted_range
not_null_stg_out_of_cash_attempted_amount_mad	not_null
not_null_stg_wallet_transactions_transaction_id	not_null
unique_stg_wallet_transactions_transaction_id	unique
not_null_stg_transactions_is_fraud	not_null
accepted_values_stg_cards_dark_web_risk__LOW__HIGH	accepted_values

G.2 Dimension and fact layer tests

Test name	Type
unique_dim_atm_atm_id / unique_dim_atm_atm_key	unique
unique_dim_card_card_id / unique_dim_card_card_key	unique
unique_dim_customer_client_id / unique_dim_customer_customer_key	unique
unique_dim_date_date_key / unique_dim_date_full_date	unique
unique_dim_merchant_merchant_id / unique_dim_merchant_merchant_key	unique
not_null_fact_card_transactions_date_key	not_null

Test name	Type
not_null_fact_atm_transactions_atm_transaction_key	not_null
unique_fact_atm_transactions_atm_transaction_key	unique
unique_fact_card_transactions_card_transaction_key	unique
unique_fact_wallet_transactions_wallet_transaction_key	unique
unique_fact_out_of_cash_events_out_of_cash_key	unique
accepted_values_fact_card_transactions_is_fraud__True__False	accepted_values
dbt_utils_accepted_range_fact_atm_transactions_amount_mad__0	accepted_range
dbt_utils_accepted_range_fact_card_transactions_amount_abs__0	accepted_range
unique_pan_customer_map_pan_key	unique

G.3 Gold mart and custom singular tests

Test name	Type
accepted_values_atm_performance_capacity_tier__LOW_CAPACITY__MEDIUM_CAPACITY__HIGH_CAPACITY	accepted_values
accepted_values_atm_performance_performance_grade__A__B__C__D	accepted_values
dbt_utils_accepted_range_atm_performance_success_rate_pct__100__0	accepted_range
accepted_values_fraud_risk_scoring_risk_level__LOW__MEDIUM__HIGH__CRITICAL	accepted_values
dbt_utils_accepted_range_fraud_risk_scoring_fraud_risk_score__100__0	accepted_range
accepted_values_replenishment_analysis_replenishment_urgency__LOW__MEDIUM__HIGH__CRITICAL	accepted_values
accepted_values_governorate_summary_macro_region__NORTH__SOUTH__EAST__OTHER	accepted_values
dbt_utils_accepted_range_customer_spending_behavior_total_spend_mad__0	accepted_range
assert_no_negative_amounts	custom singular
assert_valid_resp_codes	custom singular
assert_atm_ref_integrity	custom singular
assert_wallet_amounts_positive	custom singular

Appendix H — Sample Query Catalog

The queries below are representative examples against the Gold layer, useful both as a quick-start for a new analyst and as the basis for the cross-platform performance comparison in Section 11.

H.1 ATM replenishment urgency by region

```
SELECT
    g.region,
    r.replenishment_urgency,
    COUNT(*) AS atm_count
FROM replenishment_analysis r
JOIN dim_atm a ON r.atm_key = a.atm_key
JOIN dim_geography g ON a.region = g.region
GROUP BY g.region, r.replenishment_urgency
ORDER BY g.region, r.replenishment_urgency;
```

H.2 Top 10 customers by fraud risk score

```
SELECT
    c.client_id,
    f.fraud_risk_score,
    f.risk_level
FROM fraud_risk_scoring f
JOIN dim_customer c ON f.customer_key = c.customer_key
ORDER BY f.fraud_risk_score DESC
LIMIT 10;
```

H.3 Channel comparison — success rate by time of day

```
SELECT
    time_of_day,
    channel_code,
    success_rate_pct
FROM channel_comparison
ORDER BY time_of_day, channel_code;
```

H.4 Customer spending behavior vs. income

```
SELECT
    activity_level,
    AVG(total_spend_mad) AS avg_spend_mad,
    AVG(online_ratio_pct) AS avg_online_ratio_pct
FROM customer_spending_behavior
GROUP BY activity_level
ORDER BY avg_spend_mad DESC;
```